

class068 见识更多二维动态规划题目 笔记

视频：算法讲解068【必备】见识更多二维动态规划题目

题目1：不同的子序列

题面

给两个字符串 s 和 t ，统计在 s 的**子序列**中 t 出现的次数（子序列可以不连续，但不能改变相对次序）。答案对 1000000007 取模。

- LeetCode 115: <https://leetcode.cn/problems/distinct-subsequences/>

核心思路

二维 DP，从末尾字符讨论：

- $dp[i][j]$ ： s 前缀长度为 i 的所有子序列中，等于 t 前缀长度为 j 的子序列数量
- 可能性1（不要 $s[i-1]$ ）： $dp[i][j] = dp[i-1][j]$ ，永远可以获得
- 可能性2（要 $s[i-1]$ ）：只有当 $s[i-1] == t[j-1]$ 时， $dp[i][j] += dp[i-1][j-1]$
- 初始： $dp[i][0] = 1$ （ t 为空串，空子序列即可匹配）， $dp[0][j>0] = 0$
- 每个格子依赖**上方**和**左上角**，从左往右、从上往下填即可

坑点

- 用 `unsigned long long` 防溢出，取模版用 `int + MOD = 1000000007`
- 空间压缩：从右往左更新（因为依赖左上角），`dp[0]=1`

代码

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  class Solution {
5  public:
6      // 普通动态规划
7      // dp[i][j] : s[前缀长度为i]的所有子序列中，有多少个子序列等于t[前缀长度为j]
8      int numDistinct1(string str, string target) {
9          int n = str.length();
10         int m = target.length();
11         vector<vector<unsigned long long>> dp(n + 1, vector<unsigned long long>(m + 1,
12         0));
13         for (int i = 0; i <= n; i++) {
14             dp[i][0] = 1;
15         }
16         for (int i = 1; i <= n; i++) {
17             for (int j = 1; j <= m; j++) {
```

```

18         if (str[i - 1] == target[j - 1]) {
19             dp[i][j] += dp[i - 1][j - 1];
20         }
21     }
22 }
23 return (int)dp[n][m];
24 }
25
26 // 空间压缩
27 int numDistinct2(string str, string target) {
28     int n = str.length();
29     int m = target.length();
30     vector<unsigned long long> dp(m + 1, 0);
31     dp[0] = 1;
32     for (int i = 1; i <= n; i++) {
33         for (int j = m; j >= 1; j--) {
34             if (str[i - 1] == target[j - 1]) {
35                 dp[j] += dp[j - 1];
36             }
37         }
38     }
39     return (int)dp[m];
40 }
41
42 // 取模版本 (LeetCode 提交用)
43 int numDistinct(string s, string t) {
44     const int MOD = 1000000007;
45     int n = s.length();
46     int m = t.length();
47     vector<int> dp(m + 1, 0);
48     dp[0] = 1;
49     for (int i = 1; i <= n; i++) {
50         for (int j = m; j >= 1; j--) {
51             if (s[i - 1] == t[j - 1]) {
52                 dp[j] = (dp[j] + dp[j - 1]) % MOD;
53             }
54         }
55     }
56     return dp[m];
57 }
58 };

```

题目2：编辑距离

题面

给定两个单词 word1 和 word2，每次可以**插入**（代价 a）、**删除**（代价 b）、**替换**（代价 c）一个字符。求将 word1 转化成 word2 的最小代价。

- LeetCode 72 (特例: a=b=c=1) : <https://leetcode.cn/problems/edit-distance/>

核心思路

二维 DP，从末尾字符分类讨论：

- `dp[i][j]`：s1 前缀长度为 i 变成 s2 前缀长度为 j 的最小代价
- 若 `s1[i-1] == s2[j-1]`：直接保留 $\rightarrow dp[i][j] = dp[i-1][j-1]$
- 否则三种操作取 min：
 - 替换：`dp[i-1][j-1] + c`
 - 插入：`dp[i][j-1] + a` (s1 的 i 个搞定 s2 的 j-1 个，最后插入 s2[j-1])
 - 删除：`dp[i-1][j] + b` (s1 删掉最后一个，用前 i-1 去搞定 s2)
- 初始：`dp[i][0] = i*b`，`dp[0][j] = j*a`
- 依赖左、上、左上三个格子

坑点

- 相等时有小贪心：直接 `dp[i-1][j-1]`，其他操作一定不优于保留
- 空间压缩用 `leftUp` 变量滚过去保留左上角值
- 这是通用版 (a/b/c 可变)，LeetCode 上是 a=b=c=1 的特例

代码

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  class Solution {
5  public:
6      // LeetCode 入口：插入、删除、替换代价均为 1
7      int minDistance(string word1, string word2) {
8          return editDistance2(word1, word2, 1, 1, 1);
9      }
10
11     // 原初尝试版
12     // a : str1中插入1个字符的代价
13     // b : str1中删除1个字符的代价
14     // c : str1中改变1个字符的代价
15     int editDistance1(string str1, string str2, int a, int b, int c) {
16         int n = str1.length();
17         int m = str2.length();
18         vector<vector<int>> dp(n + 1, vector<int>(m + 1));
19         for (int i = 1; i <= n; i++) {
20             dp[i][0] = i * b;
21         }
22         for (int j = 1; j <= m; j++) {
23             dp[0][j] = j * a;
24         }
25         for (int i = 1; i <= n; i++) {
26             for (int j = 1; j <= m; j++) {
27                 if (str1[i - 1] == str2[j - 1]) {
```

```

28         dp[i][j] = dp[i - 1][j - 1];
29     } else {
30         dp[i][j] = min({dp[i - 1][j - 1] + c, dp[i][j - 1] + a, dp[i - 1]
[j] + b});
31     }
32 }
33 }
34 return dp[n][m];
35 }
36
37 // 枚举小优化版
38 int editDistance2(string str1, string str2, int a, int b, int c) {
39     int n = str1.length();
40     int m = str2.length();
41     vector<vector<int>> dp(n + 1, vector<int>(m + 1));
42     for (int i = 1; i <= n; i++) {
43         dp[i][0] = i * b;
44     }
45     for (int j = 1; j <= m; j++) {
46         dp[0][j] = j * a;
47     }
48     for (int i = 1; i <= n; i++) {
49         for (int j = 1; j <= m; j++) {
50             if (str1[i - 1] == str2[j - 1]) {
51                 dp[i][j] = dp[i - 1][j - 1];
52             } else {
53                 dp[i][j] = min({dp[i - 1][j] + b, dp[i][j - 1] + a, dp[i - 1][j -
1] + c});
54             }
55         }
56     }
57     return dp[n][m];
58 }
59
60 // 空间压缩
61 int editDistance3(string str1, string str2, int a, int b, int c) {
62     int n = str1.length();
63     int m = str2.length();
64     vector<int> dp(m + 1);
65     for (int j = 1; j <= m; j++) {
66         dp[j] = j * a;
67     }
68     for (int i = 1, leftup, backup; i <= n; i++) {
69         leftup = (i - 1) * b;
70         dp[0] = i * b;
71         for (int j = 1; j <= m; j++) {
72             backup = dp[j];
73             if (str1[i - 1] == str2[j - 1]) {
74                 dp[j] = leftup;
75             } else {
76                 dp[j] = min({dp[j] + b, dp[j - 1] + a, leftup + c});
77             }
78             leftup = backup;

```

```
79     }
80     }
81     return dp[m];
82 }
83 };
```

题目3：交错字符串

题面

给定 s_1 、 s_2 、 s_3 ，判断 s_3 是否由 s_1 和 s_2 **交错组成**（保持各自相对次序，可随意穿插）。类似洗扑克牌。

- LeetCode 97: <https://leetcode.cn/problems/interleaving-string/>

核心思路

二维 DP，讨论 s_3 最后一个字符来自谁：

- 前提： `len(s1)+len(s2) == len(s3)`，否则直接 false
- `dp[i][j]`： s_1 前缀 i 个 + s_2 前缀 j 个，能否交错出 s_3 前缀 $i+j$ 个
- 转移：
 - s_3 最后字符来自 s_1 ： `s1[i-1]==s3[i+j-1] && dp[i-1][j]`
 - s_3 最后字符来自 s_2 ： `s2[j-1]==s3[i+j-1] && dp[i][j-1]`
 - 两者或关系
- 初始： `dp[0][0]=true`，第 0 行/第 0 列看单字符串与 s_3 的前缀是否匹配
- 每个格子依赖**左**和**上**，类似最小路径和

坑点

- 这是常见错误做法：三指针外排（遇到相同字符不知选哪个）
- 必须先检查总长度是否匹配

代码

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  class Solution {
5  public:
6      // 普通动态规划
7      bool isInterleave1(string str1, string str2, string str3) {
8          int n = str1.length();
9          int m = str2.length();
10         if (n + m != (int)str3.length()) {
11             return false;
12         }
13         vector<vector<bool>>> dp(n + 1, vector<bool>(m + 1, false));
14         dp[0][0] = true;
```

```

15     for (int i = 1; i <= n; i++) {
16         if (str1[i - 1] != str3[i - 1]) {
17             break;
18         }
19         dp[i][0] = true;
20     }
21     for (int j = 1; j <= m; j++) {
22         if (str2[j - 1] != str3[j - 1]) {
23             break;
24         }
25         dp[0][j] = true;
26     }
27     for (int i = 1; i <= n; i++) {
28         for (int j = 1; j <= m; j++) {
29             dp[i][j] = (str1[i - 1] == str3[i + j - 1] && dp[i - 1][j])
30                 || (str2[j - 1] == str3[i + j - 1] && dp[i][j - 1]);
31         }
32     }
33     return dp[n][m];
34 }
35
36 // 空间压缩 (LeetCode 提交用)
37 bool isInterleave(string s1, string s2, string s3) {
38     int n = s1.length();
39     int m = s2.length();
40     if (n + m != (int)s3.length()) {
41         return false;
42     }
43     vector<bool> dp(m + 1, false);
44     dp[0] = true;
45     for (int j = 1; j <= m; j++) {
46         if (s2[j - 1] != s3[j - 1]) {
47             break;
48         }
49         dp[j] = true;
50     }
51     for (int i = 1; i <= n; i++) {
52         dp[0] = s1[i - 1] == s3[i - 1] && dp[0];
53         for (int j = 1; j <= m; j++) {
54             dp[j] = (s1[i - 1] == s3[i + j - 1] && dp[j])
55                 || (s2[j - 1] == s3[i + j - 1] && dp[j - 1]);
56         }
57     }
58     return dp[m];
59 }
60 };

```

题目4：有效涂色问题

题面

n 个格子， m 种颜色。每个格子涂一种颜色。当涂满 n 个格子后， m 种颜色**都使用了**，称为一种有效方法。求有效方法数，结果对 1000000007 取模。 $n, m \leq 5000$ 。

- 无在线测试，对数器验证

核心思路

二维 DP，分开辟新颜色和不开辟两种情况：

- $dp[i][j]$ ：前 i 个格子涂了 j 种颜色的方法数
- 不开辟新颜色：前 $i-1$ 格已凑齐 j 种，第 i 格复用 j 种之一 $\rightarrow dp[i-1][j] * j$
- 开辟新颜色：前 $i-1$ 格只有 $j-1$ 种，第 i 格选一种新颜色 $\rightarrow dp[i-1][j-1] * (m-(j-1))$
- $dp[i][j] = dp[i-1][j] * j + dp[i-1][j-1] * (m-j+1)$
- 初始： $dp[i][1] = m$ （任意一行，只能凑一种颜色就是 m 种选法）
- 依赖上方和左上角

坑点

- 中间乘法可能超 int，需用 `long long` 中转再取模
- 注意 $m-j+1$ 表示剩余可选颜色数

代码

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  const int MOD = 1000000007;
5  const int MAXN = 5001;
6
7  int dp[MAXN][MAXN];
8
9  // 正式方法
10 // 时间复杂度 $O(n * m)$ 
11 int ways2(int n, int m) {
12     //  $dp[i][j]$ :
13     // 一共有  $m$  种颜色
14     // 前  $i$  个格子涂满  $j$  种颜色的方法数
15     for (int i = 1; i <= n; i++) {
16         dp[i][1] = m;
17     }
18     for (int i = 2; i <= n; i++) {
19         for (int j = 2; j <= m; j++) {
20             dp[i][j] = (int)((long long)dp[i-1][j] * j) % MOD;
21             dp[i][j] = (int)((long long)dp[i-1][j-1] * (m-j+1) + dp[i][j]) %
MOD);
22         }
```

```

23     }
24     return dp[n][m];
25 }

```

题目5：删除至少几个字符可以变成另一个字符串的子串

题面

给定 s_1 和 s_2 ，求 s_1 至少删除多少个字符可以变成 s_2 的**子串**（子串必须连续）。

- 无在线测试，对数器验证

核心思路

二维 DP，定义很巧妙：

- $dp[i][j]$ ： s_1 前缀 i 个字符，至少删除多少个字符，能变成 s_2 前缀 j 个字符的**任意后缀串**
- 若 $s_1[i-1] == s_2[j-1]$ ：保留，问题变为 $dp[i-1][j-1]$
- 若不等：删掉 $s_1[i-1]$ ， $1 + dp[i-1][j]$
- 初始： $dp[i][0] = i$ （ s_2 为空，删掉 s_1 全部）， $dp[0][j] = 0$ （ s_1 为空天生是任意后缀串）
- 最终答案：**最后一行的最小值**（因为 s_1 整体变成的可能是 s_2 以任意位置结尾的后缀串）

坑点

- 返回值是 $\min(dp[n][0..m])$ 而非 $dp[n][m]$ —— 因为变成的子串可能以 s_2 任意位置结尾
- 依赖**上方**和**左上**，常规填表即可

代码

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  // 正式方法，动态规划
5  int minDelete2(string str1, string str2) {
6      int n = str1.length();
7      int m = str2.length();
8      // dp[len1][len2] :
9      // s1[前缀长度为i]至少删除多少字符，可以变成s2[前缀长度为j]的任意后缀串
10     vector<vector<int>> dp(n + 1, vector<int>(m + 1));
11     for (int i = 1; i <= n; i++) {
12         dp[i][0] = i;
13         for (int j = 1; j <= m; j++) {
14             if (str1[i - 1] == str2[j - 1]) {
15                 dp[i][j] = dp[i - 1][j - 1];
16             } else {
17                 dp[i][j] = dp[i - 1][j] + 1;
18             }
19         }
20     }

```



```
21     int ans = INT_MAX;
22     for (int j = 0; j <= m; j++) {
23         ans = min(ans, dp[n][j]);
24     }
25     return ans;
26 }
```

class069 从递归入手三维动态规划 笔记

视频：算法讲解069【必备】从递归入手三维动态规划

题目1：一和零（多维费用背包）

题面

给定二进制字符串数组 strs 和两个整数 m、n。选出最大的子集，使得子集中**最多**有 m 个 0 和 n 个 1。每个字符串由 '0' 和 '1' 组成。

- LeetCode 474: <https://leetcode.cn/problems/ones-and-zeroes/>

核心思路

三维 DP / 多维费用 0-1 背包，每个字符串要或不要：

- `dp[i][z][o]`：从 i 开始往后自由选择，零不超过 z、一不超过 o，最多选几个字符串
- 不要当前： `dp[i+1][z][o]`
- 要当前（需 $\text{zeros} \leq z$ 且 $\text{ones} \leq o$ ）： `1 + dp[i+1][z-zeros][o-ones]`
- 三维表中每层只依赖上一层**，层之间无依赖 → 从上往下填
- 空间压缩：二维数组 `dp[z][o]`，z 和 o 都**从大到小**更新（因为依赖上一层的 z-zeros, o-ones）

坑点

- 空间压缩时 z 和 o 都要从大到小遍历，否则会读到本层刚更新的值
- 每个字符串的 zeros/ones 通过 `zerosAndOnes()` 统计到全局变量

代码

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  class Solution {
5  public:
6      int zeros, ones;
7
8      // 统计一个字符串中0和1的数量
9      void zerosAndOnes(const string& str) {
10         zeros = 0;
11         ones = 0;
```

```

12     for (char c : str) {
13         if (c == '0') {
14             zeros++;
15         } else {
16             ones++;
17         }
18     }
19 }
20
21 // 递归尝试
22 int f1(vector<string>& strs, int i, int z, int o) {
23     if (i == (int)strs.size()) {
24         return 0;
25     }
26     int p1 = f1(strs, i + 1, z, o);
27     int p2 = 0;
28     zerosAndOnes(strs[i]);
29     if (zeros <= z && ones <= o) {
30         p2 = 1 + f1(strs, i + 1, z - zeros, o - ones);
31     }
32     return max(p1, p2);
33 }
34
35 // 记忆化搜索
36 int findMaxForm2(vector<string>& strs, int m, int n) {
37     int len = strs.size();
38     vector<vector<vector<int>>> dp(len, vector<vector<int>>(m + 1, vector<int>(n +
39 1, -1)));
40     return f2(strs, 0, m, n, dp);
41 }
42
43 int f2(vector<string>& strs, int i, int z, int o, vector<vector<vector<int>>>& dp)
44 {
45     if (i == (int)strs.size()) {
46         return 0;
47     }
48     if (dp[i][z][o] != -1) {
49         return dp[i][z][o];
50     }
51     int p1 = f2(strs, i + 1, z, o, dp);
52     int p2 = 0;
53     zerosAndOnes(strs[i]);
54     if (zeros <= z && ones <= o) {
55         p2 = 1 + f2(strs, i + 1, z - zeros, o - ones, dp);
56     }
57     int ans = max(p1, p2);
58     dp[i][z][o] = ans;
59     return ans;
60 }
61
62 // 三维动态规划
63 int findMaxForm3(vector<string>& strs, int m, int n) {
64     int len = strs.size();

```

```

63     vector<vector<vector<int>>> dp(len + 1, vector<vector<int>>(m + 1, vector<int>
(n + 1, 0)));
64     for (int i = len - 1; i >= 0; i--) {
65         zerosAndOnes(strs[i]);
66         for (int z = 0; z <= m; z++) {
67             for (int o = 0; o <= n; o++) {
68                 int p1 = dp[i + 1][z][o];
69                 int p2 = 0;
70                 if (zeros <= z && ones <= o) {
71                     p2 = 1 + dp[i + 1][z - zeros][o - ones];
72                 }
73                 dp[i][z][o] = max(p1, p2);
74             }
75         }
76     }
77     return dp[0][m][n];
78 }
79
80 // 空间压缩 (LeetCode 提交用)
81 int findMaxForm(vector<string>& strs, int m, int n) {
82     vector<vector<int>> dp(m + 1, vector<int>(n + 1, 0));
83     for (const string& s : strs) {
84         zerosAndOnes(s);
85         for (int z = m; z >= zeros; z--) {
86             for (int o = n; o >= ones; o--) {
87                 dp[z][o] = max(dp[z][o], 1 + dp[z - zeros][o - ones]);
88             }
89         }
90     }
91     return dp[m][n];
92 }
93 };

```

题目2：盈利计划（多维费用背包）

题面

n 名员工，第 i 项工作消耗 $group[i]$ 人、产生 $profit[i]$ 利润。员工不能同时参与多项工作。求选出一些工作使得**员工总数** $\leq n$ 且**利润** $\geq minProfit$ 的方案数。答案取模 1000000007。

- LeetCode 879: <https://leetcode.cn/problems/profitable-schemes/>

核心思路

三维 DP / 多维费用 0-1 背包，利润维度做特殊处理：

- $dp[i][r][s]$ ：从第 i 号工作开始，员工额度 r 、还需利润 s 达标，有多少种方案
- base case:
 - $r \leq 0$ 且 $s \leq 0 \rightarrow 1$ (已达标)
 - i 越界且 $s \leq 0 \rightarrow 1$

- 否则 $\rightarrow 0$
- 转移：不要当前 + 要当前（人够的前提下），和题目 1 一样
- **关键优化**：利润要求到了负数等同于 0 ($\max(0, s - \text{profit}[i])$)，防止 DP 表下标越界
- 空间压缩：二维 $\text{dp}[r][s]$ ， r 从大到小、 s 从大到小更新

坑点

- $\max(0, s - \text{profit}[i])$ 是灵魂操作：利润达标后多出来的利润没区别，统一压到 0
- 初始 $\text{dp}[r][0] = 1$ ：利润已达标的时候方案数为 1

代码

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  class Solution {
5  public:
6      const int MOD = 1000000007;
7
8      // 递归尝试
9      int f1(vector<int>& g, vector<int>& p, int i, int r, int s) {
10         if (r <= 0) {
11             return s <= 0 ? 1 : 0;
12         }
13         if (i == (int)g.size()) {
14             return s <= 0 ? 1 : 0;
15         }
16         int p1 = f1(g, p, i + 1, r, s);
17         int p2 = 0;
18         if (g[i] <= r) {
19             p2 = f1(g, p, i + 1, r - g[i], s - p[i]);
20         }
21         return p1 + p2;
22     }
23
24     // 记忆化搜索
25     int profitableSchemes2(int n, int minProfit, vector<int>& group, vector<int>&
profit) {
26         int m = group.size();
27         vector<vector<vector<int>>> dp(m, vector<vector<int>>(n + 1, vector<int>
(minProfit + 1, -1)));
28         return f2(group, profit, 0, n, minProfit, dp);
29     }
30
31     int f2(vector<int>& g, vector<int>& p, int i, int r, int s,
vector<vector<vector<int>>>& dp) {
32         if (r <= 0) {
33             return s == 0 ? 1 : 0;
34         }
35         if (i == (int)g.size()) {
36             return s == 0 ? 1 : 0;

```

```

37     }
38     if (dp[i][r][s] != -1) {
39         return dp[i][r][s];
40     }
41     int p1 = f2(g, p, i + 1, r, s, dp);
42     int p2 = 0;
43     if (g[i] <= r) {
44         p2 = f2(g, p, i + 1, r - g[i], max(0, s - p[i]), dp);
45     }
46     int ans = (p1 + p2) % MOD;
47     dp[i][r][s] = ans;
48     return ans;
49 }
50
51 // 空间压缩版动态规划 (LeetCode 提交用)
52 int profitableSchemes(int n, int minProfit, vector<int>& group, vector<int>&
profit) {
53     vector<vector<int>>> dp(n + 1, vector<int>(minProfit + 1, 0));
54     for (int r = 0; r <= n; r++) {
55         dp[r][0] = 1;
56     }
57     int m = group.size();
58     for (int i = m - 1; i >= 0; i--) {
59         for (int r = n; r >= 0; r--) {
60             for (int s = minProfit; s >= 0; s--) {
61                 int p1 = dp[r][s];
62                 int p2 = group[i] <= r ? dp[r - group[i]][max(0, s - profit[i])] :
0;
63                 dp[r][s] = (p1 + p2) % MOD;
64             }
65         }
66     }
67     return dp[n][minProfit];
68 }
69 };

```

题目3：骑士在棋盘上的概率

题面

$n \times n$ 棋盘，骑士从 (row, col) 出发走 k 步，每步等概率走 8 个"马走日"方向之一。一旦出棋盘就死亡。求 k 步后仍在棋盘上的概率。

- LeetCode 688: <https://leetcode.cn/problems/knight-probability-in-chessboard/>

核心思路

三维 DP (记忆化搜索即可, 返回 double 类型) :

- `dp[i][j][k]`: 从 (i,j) 出发还剩 k 步, 最后在棋盘上的概率
- 越界 $\rightarrow 0$

- $k=0 \rightarrow 1$ (不用走且还在棋盘内)
- 8 个方向等概率各 $1/8 \rightarrow$ 返回 8 个方向的概率之和除以 8
- K 依赖 K-1 层, 三维表按层从低到高填

坑点

- 返回 double, DP 表用 double 或 -1 标记未计算
- 空间压缩可以用两张二维表交替 (K 层只用 K-1 层), 但记忆化搜索已足够快

代码

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  class Solution {
5  public:
6      // 记忆化搜索
7      // 从(i,j)出发还有k步要走, 返回最后在棋盘上的概率
8      double f(int n, int i, int j, int k, vector<vector<vector<double>>>& dp) {
9          if (i < 0 || i >= n || j < 0 || j >= n) {
10             return 0;
11         }
12         if (dp[i][j][k] != -1) {
13             return dp[i][j][k];
14         }
15         double ans = 0;
16         if (k == 0) {
17             ans = 1;
18         } else {
19             ans += (f(n, i - 2, j + 1, k - 1, dp) / 8);
20             ans += (f(n, i - 1, j + 2, k - 1, dp) / 8);
21             ans += (f(n, i + 1, j + 2, k - 1, dp) / 8);
22             ans += (f(n, i + 2, j + 1, k - 1, dp) / 8);
23             ans += (f(n, i + 2, j - 1, k - 1, dp) / 8);
24             ans += (f(n, i + 1, j - 2, k - 1, dp) / 8);
25             ans += (f(n, i - 1, j - 2, k - 1, dp) / 8);
26             ans += (f(n, i - 2, j - 1, k - 1, dp) / 8);
27         }
28         dp[i][j][k] = ans;
29         return ans;
30     }
31
32     double knightProbability(int n, int k, int row, int column) {
33         vector<vector<vector<double>>> dp(n, vector<vector<double>>(n, vector<double>(k
+ 1, -1)));
34         return f(n, row, column, k, dp);
35     }
36 };

```

题目4：矩阵中和能被 K 整除的路径

题面

$n \times m$ 矩阵，从 (0,0) 出发只能向右或向下走到 (n-1,m-1)。求路径和能被 k 整除的路径数，结果取模 1000000007。

- LeetCode 2435: <https://leetcode.cn/problems/paths-in-matrix-whose-sum-is-divisible-by-k/>

核心思路

三维 DP，余数做第三维：

- $dp[i][j][r]$ ：从 (i,j) 出发到右下角，路径和模 k 余 r 的路径数
- 当前格值模 $k = cur = grid[i][j] \% k$
- 后续需要凑的余数： $need = (k + r - cur) \% k$ （核心公式）
- base case：右下角 $dp[n-1][m-1][grid[n-1][m-1]\%k] = 1$
- 转移： $dp[i][j][r] = dp[i+1][j][need] + dp[i][j+1][need]$
- 从右下往左上填：先填最后一行/列，再填中间

坑点

- $need = (k + r - cur) \% k$ 这个公式是关键，保证非负
- 三维 DP 规模是 $n \times m \times k$ ，每个格子 $O(1)$ 转移（没有枚举）

代码

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  class Solution {
5  public:
6      const int MOD = 1000000007;
7
8      // 记忆化搜索
9      int numberOfPaths2(vector<vector<int>>& grid, int k) {
10         int n = grid.size();
11         int m = grid[0].size();
12         vector<vector<vector<int>>> dp(n, vector<vector<int>>(m, vector<int>(k, -1)));
13         return f2(grid, n, m, k, 0, 0, 0, dp);
14     }
15
16     int f2(vector<vector<int>>& grid, int n, int m, int k, int i, int j, int r,
17 vector<vector<vector<int>>>& dp) {
18         if (i == n - 1 && j == m - 1) {
19             return grid[i][j] % k == r ? 1 : 0;
20         }
21         if (dp[i][j][r] != -1) {
22             return dp[i][j][r];
23         }
24         int need = (k + r - grid[i][j] % k) % k;
```

```

24     int ans = 0;
25     if (i + 1 < n) {
26         ans = f2(grid, n, m, k, i + 1, j, need, dp);
27     }
28     if (j + 1 < m) {
29         ans = (ans + f2(grid, n, m, k, i, j + 1, need, dp)) % MOD;
30     }
31     dp[i][j][r] = ans;
32     return ans;
33 }
34
35 // 动态规划 (LeetCode 提交用)
36 int numberOfPaths(vector<vector<int>>& grid, int k) {
37     int n = grid.size();
38     int m = grid[0].size();
39     vector<vector<vector<int>>> dp(n, vector<vector<int>>(m, vector<int>(k, 0)));
40     dp[n - 1][m - 1][grid[n - 1][m - 1] % k] = 1;
41     // 最后一列
42     for (int i = n - 2; i >= 0; i--) {
43         for (int r = 0; r < k; r++) {
44             dp[i][m - 1][r] = dp[i + 1][m - 1][(k + r - grid[i][m - 1] % k) % k];
45         }
46     }
47     // 最后一行
48     for (int j = m - 2; j >= 0; j--) {
49         for (int r = 0; r < k; r++) {
50             dp[n - 1][j][r] = dp[n - 1][j + 1][(k + r - grid[n - 1][j] % k) % k];
51         }
52     }
53     for (int i = n - 2; i >= 0; i--) {
54         for (int j = m - 2; j >= 0; j--) {
55             for (int r = 0; r < k; r++) {
56                 int need = (k + r - grid[i][j] % k) % k;
57                 dp[i][j][r] = dp[i + 1][j][need];
58                 dp[i][j][r] = (dp[i][j][r] + dp[i][j + 1][need]) % MOD;
59             }
60         }
61     }
62     return dp[0][0][0];
63 }
64 };

```

题目5：扰乱字符串

题面

给定等长字符串 s_1 和 s_2 ，判断 s_2 是否是 s_1 的扰乱字符串。扰乱规则：可将字符串任意分割成两个非空子串，每层可选择交换或不交换左右两部分，递归进行。

- LeetCode 87: <https://leetcode.cn/problems/scramble-string/>

核心思路

三维 DP + 区间 DP 思想, $O(n^4)$ (n^3 格子 $\times$ n 枚举) :

- `dp[l1][l2][len]`: `s1` 从 `l1` 开始、`s2` 从 `l2` 开始, 长度均为 `len`, 是否互为扰乱串
- 初始: `len=1` 时 `dp[l1][l2][1] = s1[l1]==s2[l2]`
- 转移——枚举分割点 `k` ($1 \leq k < len$) :
 - 不交错: `dp[l1][l2][k] && dp[l1+k][l2+k][len-k]`
 - 交错: `dp[l1][l2+len-k][k] && dp[l1+k][l2][len-k]`
 - 任一种成立即 `true`
- 枚举 `k` 有 $O(n)$ 开销, 无法优化
- 从 `len=2` 往上递推, 注意边界: `l1 ≤ n-len, l2 ≤ n-len`

坑点

- 字符种类和数量都相同 $\neq$ 一定是扰乱串 (反例: `abcd` vs `cadb`)
- `len` 从 2 到 `n` 递增, `l1/l2` 不能越界 (`l1 ≤ n-len`)
- 记忆化搜索用 `int` 三态 (`0`=未算, `-1`=false, `1`=true), 因为 `bool` 无法区分"没算过"

代码

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  class Solution {
5  public:
6      // 记忆化搜索
7      // dp[l1][l2][len] : 0 -> 没展开过, -1 -> false, 1 -> true
8      bool isScramble3(string s1, string s2) {
9          int n = s1.length();
10         vector<vector<vector<int>>> dp(n, vector<vector<int>>(n, vector<int>(n + 1, 0)));
11         return f3(s1, s2, 0, 0, n, dp);
12     }
13
14     bool f3(const string& s1, const string& s2, int l1, int l2, int len,
15 vector<vector<vector<int>>>& dp) {
16         if (len == 1) {
17             return s1[l1] == s2[l2];
18         }
19         if (dp[l1][l2][len] != 0) {
20             return dp[l1][l2][len] == 1;
21         }
22         bool ans = false;
23         // 不交错
24         for (int k = 1; k < len; k++) {
25             if (f3(s1, s2, l1, l2, k, dp) && f3(s1, s2, l1 + k, l2 + k, len - k, dp)) {
26                 ans = true;
27             }
28         }
29         // 交错
30         if (f3(s1, s2, l1, l2 + len - k, k, dp) && f3(s1, s2, l1 + k, l2, len - k, dp)) {
31             ans = true;
32         }
33         dp[l1][l2][len] = ans;
34         return ans;
35     }
36 }
```

```

26         break;
27     }
28 }
29 // 交错
30 if (!ans) {
31     for (int i = l1 + 1, j = l2 + len - 1, k = 1; k < len; i++, j--, k++) {
32         if (f3(s1, s2, l1, j, k, dp) && f3(s1, s2, i, l2, len - k, dp)) {
33             ans = true;
34             break;
35         }
36     }
37 }
38 dp[l1][l2][len] = ans ? 1 : -1;
39 return ans;
40 }
41
42 // 严格位置依赖的动态规划 (LeetCode 提交用)
43 bool isScramble(string s1, string s2) {
44     int n = s1.length();
45     vector<vector<vector<bool>>> dp(n, vector<vector<bool>>(n, vector<bool>(n + 1,
false)));
46     // len = 1
47     for (int l1 = 0; l1 < n; l1++) {
48         for (int l2 = 0; l2 < n; l2++) {
49             dp[l1][l2][1] = s1[l1] == s2[l2];
50         }
51     }
52     for (int len = 2; len <= n; len++) {
53         for (int l1 = 0; l1 <= n - len; l1++) {
54             for (int l2 = 0; l2 <= n - len; l2++) {
55                 for (int k = 1; k < len; k++) {
56                     if (dp[l1][l2][k] && dp[l1 + k][l2 + k][len - k]) {
57                         dp[l1][l2][len] = true;
58                         break;
59                     }
60                 }
61                 if (!dp[l1][l2][len]) {
62                     for (int i = l1 + 1, j = l2 + len - 1, k = 1; k < len; i++, j-
-, k++) {
63                         if (dp[l1][j][k] && dp[i][l2][len - k]) {
64                             dp[l1][l2][len] = true;
65                             break;
66                         }
67                     }
68                 }
69             }
70         }
71     }
72     return dp[0][0][n];
73 }
74 };

```

class070 子数组最大累加和问题与扩展（上）

B站视频：[算法讲解070](#)

前置知识：class051 二分答案法、class066 一维动态规划

01 子数组最大累加和 — LeetCode 53

题面：给定整数数组，求非空连续子数组的最大累加和。

核心思路： $dp[i]$ = 子数组必须以*i*结尾往左延伸的最大累加和。两种选择：自己待着 $nums[i]$ ，或往左扩 $dp[i-1]+nums[i]$ 。空间压缩只需一个 pre 变量滚动。

附加问题——同时返回开头的 $left$ 、结尾的 $right$ 、累加和 sum ： $pre \geq 0$ 时不换开头， $pre < 0$ 时换开头 $l=r$ ， $pre > sum$ 时更新三个全局变量。 pre 初始为 INT_MIN 保证0位置正确初始化。

坑点：无。

代码：

```
1  int maxSubArray(vector<int>& nums) {
2      int n = nums.size(), ans = nums[0];
3      for (int i = 1, pre = nums[0]; i < n; i++) {
4          pre = max(nums[i], pre + nums[i]);
5          ans = max(ans, pre);
6      }
7      return ans;
8  }
9  // 附加：同时求 left, right, sum
10 int left, right, sum;
11 void extra(vector<int>& nums) {
12     sum = INT_MIN;
13     for (int l = 0, r = 0, pre = INT_MIN; r < (int)nums.size(); r++) {
14         if (pre >= 0) pre += nums[r];
15         else { pre = nums[r]; l = r; }
16         if (pre > sum) { sum = pre; left = l; right = r; }
17     }
18 }
```

02 不选相邻元素的最大累加和（打家劫舍） — LeetCode 198

题面：数组中可以随意选数，但不能选相邻的。求最大累加和。

核心思路：与题目1不同，这里是子序列。 $dp[i]$ = $[0..i]$ 范围上随意选但不能相邻的最大累加和。转移：①不要 $nums[i] \rightarrow dp[i-1]$ ；②要 $nums[i] \rightarrow \max(nums[i], dp[i-2]+nums[i])$ 。空间压缩用 $prepre$ ($dp[i-2]$)和 pre ($dp[i-1]$)两变量滚动。

坑点： $n=1$ 和 $n=2$ 要单独处理。

代码：

```

1  int rob(vector<int>& nums) {
2      int n = nums.size();
3      if (n == 1) return nums[0];
4      if (n == 2) return max(nums[0], nums[1]);
5      int prepre = nums[0], pre = max(nums[0], nums[1]);
6      for (int i = 2, cur; i < n; i++) {
7          cur = max(pre, max(nums[i], prepre + nums[i]));
8          prepre = pre; pre = cur;
9      }
10     return pre;
11 }

```

03 环形子数组最大累加和 — LeetCode 918

题面：数组首尾相连（环形），求非空连续子数组最大累加和。

核心思路：答案分两种：①不被隔断→普通子数组最大累加和 `maxSum`；②被隔断（开头取一段+结尾取一段）= `all - minSum`（整体减掉中间最差部分）。同时维护 `maxpre` 和 `minpre`。特例： `all == minSum` 说明整块被抠掉（空数组不合法），只返回 `maxSum`。

坑点： `all == minSum` 时不能选 `all - minSum`（相当于空数组）。

代码：

```

1  int maxSubarraySumCircular(vector<int>& nums) {
2      int n = nums.size(), all = nums[0], maxsum = nums[0], minsum = nums[0];
3      for (int i = 1, maxpre = nums[0], minpre = nums[0]; i < n; i++) {
4          all += nums[i];
5          maxpre = max(nums[i], nums[i] + maxpre);
6          maxsum = max(maxsum, maxpre);
7          minpre = min(nums[i], nums[i] + minpre);
8          minsum = min(minsum, minpre);
9      }
10     return all == minsum ? maxsum : max(maxsum, all - minsum);
11 }

```

04 环形数组 + 不能选相邻元素（打家劫舍 II） — LeetCode 213

题面：房屋围成一圈（首尾相邻），不能偷相邻的，求最大金额。

核心思路：按 `nums[0]` 要不要分两类：①不要 `nums[0]` → 在 `[1, n-1]` 上跑普通打家劫舍；②要 `nums[0]` → `nums[0] + [2, n-2]` 范围上跑普通打家劫舍。取 `max`。

坑点： `n=1` 要单独处理。

代码：

```

1  int best(vector<int>& nums, int l, int r) {
2      if (l > r) return 0;
3      if (l == r) return nums[l];

```

```

4     if (l + 1 == r) return max(nums[l], nums[r]);
5     int prepre = nums[l], pre = max(nums[l], nums[l + 1]);
6     for (int i = l + 2, cur; i <= r; i++) {
7         cur = max(pre, nums[i] + max(0, prepre));
8         prepre = pre; pre = cur;
9     }
10    return pre;
11 }
12 int rob(vector<int>& nums) {
13     int n = nums.size();
14     if (n == 1) return nums[0];
15     return max(best(nums, 1, n - 1), nums[0] + best(nums, 2, n - 2));
16 }

```

05 打家劫舍 IV — LeetCode 2560

题面：窃取能力=能偷的房间金额上限。必须偷至少k间（不能相邻），问最小能力值是多少。

核心思路：二分答案法。能力范围 $[\min(\text{nums}), \max(\text{nums})]$ 。判断函数 `mostRob(ability)`：能力定了，贪心求最多偷几间——遇到能偷的尽早偷然后跳过相邻。若 `mostRob(mid) >= k` 则记录答案并搜左边更小能力。

坑点：贪心成立的原因是收益都是+1（偷一间），不是加金额。尽早偷能给后面留更大范围。

代码：

```

1  int check(vector<int> &nums, int n, int ab){
2      if(n==1)
3          return nums[0] <= ab ? 1 : 0;
4      if(n==2)
5          return (nums[0] <= ab || nums[1] <= ab) ? 1 : 0;
6      int pre2 = nums[0] <= ab ? 1 : 0;
7      int pre1 = (nums[0] <= ab || nums[1] <= ab) ? 1 : 0;
8      for (int i = 2, cur; i < n; ++i){
9          cur = max(pre1, pre2 + (nums[i] <= ab ? 1 : 0));
10         pre2 = pre1;
11         pre1 = cur;
12     }
13     return pre1;
14 }
15
16 int minCapability(vector<int> &nums, int k){
17     int n = nums.size();
18     int l = nums[0], r = nums[n-1];
19     for (int i = 0; i < n; ++i){
20         l = nums[i] < l ? nums[i] : l;
21         r = nums[i] > r ? nums[i] : r;
22     }
23     int m, ans = 0;
24     while(l <= r){
25         m = (l + r) / 2;
26         if(check(nums, n, m) >= k){
27             ans = m;

```

```

28         r = m - 1;
29     }else{
30         l = m + 1;
31     }
32 }
33 return ans;
34 }

```

06 子矩阵最大累加和 — 面试题 17.24

题面：二维数组，找子矩阵最大累加和，返回左上角和右下角坐标。

核心思路：压缩数组。枚举起始行 `up` 和结束行 `down` ($O(n^2)$)，把 `[up..down]` 行按列求和压缩成一维数组，在上面跑子数组最大累加和（边生成边判断），同步记录全局最大时的左上角 (`up, l`) 和右下角 (`down, r`)。任何子矩阵必从某行开始到某行结束，枚举了所有组合。

坑点：时间复杂度 $O(n^2 \cdot m)$ ，最坏 $O(n^3)$ ，已是最优解。空间 $O(m)$ 。

代码：

```

1  vector<int> getMaxMatrix(vector<vector<int>>& grid) {
2      int n = grid.size(), m = grid[0].size();
3      int maxSum = INT_MIN, a = 0, b = 0, c = 0, d = 0;
4      vector<int> nums(m);
5      for (int up = 0; up < n; up++) {
6          fill(nums.begin(), nums.end(), 0);
7          for (int down = up; down < n; down++) {
8              for (int l = 0, r = 0, pre = INT_MIN; r < m; r++) {
9                  nums[r] += grid[down][r];
10                 if (pre >= 0) pre += nums[r];
11                 else { pre = nums[r]; l = r; }
12                 if (pre > maxSum) {
13                     maxSum = pre; a = up; b = l; c = down; d = r;
14                 }
15             }
16         }
17     }
18     return {a, b, c, d};
19 }

```

class071 子数组最大累加和问题与扩展（下）

B站视频：[算法讲解071](#)

前置知识：class005 对数器、class046 前缀信息技巧、class054 单调队列

01 乘积最大子数组 — LeetCode 152

题面：整数数组，求乘积最大的非空连续子数组的乘积。

核心思路：与累加和版本的关键区别：有负数→负负得正。每步需同时维护以i结尾的**最大乘积**和**最小乘积**。三种可能：自己待着 `nums[i]`、乘之前最大 `maxVal*nums[i]`、乘之前最小 `minVal*nums[i]`。分别取max和min赋给 `curmax`和`curmin`，滚动更新。

坑点：用 `double` 防中间溢出，答案转 `int`。

代码：

```
1  int maxProduct(vector<int>& nums) {
2      double ans = nums[0], minVal = nums[0], maxVal = nums[0], curmin, curmax;
3      for (int i = 1; i < (int)nums.size(); i++) {
4          curmin = min({(double)nums[i], minVal * nums[i], maxVal * nums[i]});
5          curmax = max({(double)nums[i], minVal * nums[i], maxVal * nums[i]});
6          minVal = curmin; maxVal = curmax;
7          ans = max(ans, maxVal);
8      }
9      return (int)ans;
10 }
```

02 子序列累加和被7整除的最大累加和

题面：非负数组，任选数字组成子序列，累加和必须被7整除，返回最大累加和。

核心思路：dp第二位用**余数**而非累加和。`dp[i][j]` = 前i个数形成的子序列累加和%7=j的最大累加和（-1表示不存在）。转移：不要 `nums[i-1]` → `dp[i-1][j]`；要→找 `need=(7+j-cur)%7`，`dp[i-1][need]+x`。初始化 `dp[0][0]=0`（空集），其余-1。余数只有7种， $O(7n)=O(n)$ 。

坑点：空集的累加和为0，模7为0，所以 `dp[0][0]=0` 有效；`dp[0][1..6]=-1` 无效。

代码：

```
1  int maxSum2(vector<int>& nums) {
2      int n = nums.size();
3      dp[0][0] = 0;
4      for (int j = 1; j < 7; j++) dp[0][j] = -1;
5      for (int i = 1, x, cur, need; i <= n; i++) {
6          x = nums[i - 1]; cur = nums[i - 1] % 7;
7          for (int j = 0; j < 7; j++) {
8              dp[i][j] = dp[i - 1][j];
9              need = (7 + j - cur) % 7;
10             if (dp[i - 1][need] != -1)
11                 dp[i][j] = max(dp[i][j], dp[i - 1][need] + x);
12         }
13     }
14     return dp[n][0];
15 }
```

03 魔法卷轴 — 两次清零机会的最大累加和

题面：数组有正负零，最多2次机会把任意连续一段全变0，求操作后数组整体的最大累加和。

核心思路：三种情况：①0次→全体求和；②1次→ $pdp[i] = 0 \sim i$ 必用1次整体最大，转移为 $\max(pdp[i-1] + nums[i], \maxPresum)$ ，其中 $\maxPresum$ 是所有前缀和+0的最大值；③2次→枚举划分点 i ， $pdp[i-1] + sdp[i]$ 。 sdp 从右往左对称求。

坑点： $\maxPresum$ 初始要考虑0（一个数也不要时前缀和为0，可以全清掉）。

代码：

```
1 int maxSum2(vector<int>& nums) {
2     int n = nums.size();
3     if (n == 0) return 0;
4     int p1 = 0; for (int num : nums) p1 += num;
5     int sum = nums[0], maxPresum = max(0, nums[0]);
6     for (int i = 1; i < n; i++) {
7         pdp[i] = max(pdp[i - 1] + nums[i], maxPresum);
8         sum += nums[i];
9         maxPresum = max(maxPresum, sum);
10    }
11    int p2 = pdp[n - 1];
12    sdp[n - 1] = 0; sum = nums[n - 1]; maxPresum = max(0, sum);
13    for (int i = n - 2; i >= 0; i--) {
14        sdp[i] = max(nums[i] + sdp[i + 1], maxPresum);
15        sum += nums[i]; maxPresum = max(maxPresum, sum);
16    }
17    int p3 = INT_MIN;
18    for (int i = 1; i < n; i++) p3 = max(p3, pdp[i - 1] + sdp[i]);
19    return max(p1, max(p2, p3));
20 }
```

04 三个无重叠子数组的最大和 — LeetCode 689

题面：给定数组和整数 k ，选3段长度为 k 、不重叠的子数组，使总和最大，返回字典序最小的3个起始下标。

核心思路：三个辅助结构：① $sums[i]$ = 以 i 开头长度 k 的累加和（滑动窗口）；② $prefix[i]$ = $0 \sim i$ 范围长度 k 子数组的最大累加和的开头；③ $suffix[i]$ = $i \sim n-1$ 范围同理。枚举中间子数组 i ，左侧最优 = $prefix[i-1]$ ，右侧最优 = $suffix[i+k]$ ，总额 = $sums[p] + sums[i] + sums[s]$ 。字典序处理： $prefix$ 新 > 老才更新（相等保留小的）， $suffix$ 新 >= 老就更新（相等选新，更靠左）。

坑点： $prefix$ 和 $suffix$ 的等号方向不同，都是为保证字典序最小。

代码：

```
1 vector<int> maxSumOfThreeSubarrays(vector<int>& nums, int k) {
2     int n = nums.size();
3     for (int l = 0, r = 0, sum = 0; r < n; r++) {
4         sum += nums[r];
5         if (r - l + 1 == k) { sums[l] = sum; sum -= nums[l]; l++; }
6     }
7     for (int l = 1, r = k; r < n; l++, r++) {
8         if (sums[l] > sums[prefix[r - 1]]) prefix[r] = l;
9         else prefix[r] = prefix[r - 1];
10    }
```



```

11     suffix[n - k] = n - k;
12     for (int l = n - k - 1; l >= 0; l--) {
13         if (sums[l] >= sums[suffix[l + 1]]) suffix[l] = l;
14         else suffix[l] = suffix[l + 1];
15     }
16     int a = 0, b = 0, c = 0, maxSum = 0;
17     for (int p, s, i = k, j = 2 * k - 1, sum; j < n - k; i++, j++) {
18         p = prefix[i - 1]; s = suffix[j + 1];
19         sum = sums[p] + sums[i] + sums[s];
20         if (sum > maxSum) { maxSum = sum; a = p; b = i; c = s; }
21     }
22     return {a, b, c};
23 }

```

05 翻转一次子数组最大累加和

题面： 随意选连续一段翻转（逆序），返回翻转后子数组最大累加和。

核心思路：

反转后得到最大累加和，这个子数组累加和的区间一定有一部分是翻转得来的，很容易排除翻转区间都在子数组内或者子数组完全被包含在翻转区间内的可能。

0..i-1 i...n-1

用maxend得到前面的最大子数组累加和，这一部分用于翻转到i前面

sta[i]表示必须以i开头的最大子数组累加和，这部分是没有翻转的。

枚举每一个i位置就能得到最终答案。

坑点：

比如 [-1,-6,-5,4,-2,-3]

最大是4，由maxend得到，对应的是无有效翻转的情况（只翻转一个数字），要加一层判断

代码：

```

1  int maxSumReverse2(vector<int>& nums) {
2      int n = nums.size();
3      start[n - 1] = nums[n - 1];
4      for (int i = n - 2; i >= 0; i--)
5          start[i] = max(nums[i], nums[i] + start[i + 1]);
6      int ans = start[0];
7      int end = nums[0], maxEnd = nums[0];
8      for (int i = 1; i < n; i++) {
9          ans = max(ans, maxEnd + start[i]);
10         end = max(nums[i], end + nums[i]);
11         maxEnd = max(maxEnd, end);
12     }
13     ans = max(ans, maxEnd);
14     return ans;
15 }

```

06 删掉1个数后长度为k的子数组最大累加和

题面：必须删1个数形成新数组，在新数组中找长度为k的子数组最大累加和。删哪个随意。

核心思路：巧妙转化：原问题=在原数组中考察每个**长度为k+1的窗口**，**窗口累加和 - 窗口内最小值**。无论删的数在答案内部还是外部，该转化都能覆盖。实现：滑动窗口维护累加和+单调队列维护窗口最小值。

坑点： $n \leq k$ 时删完不够k个，返回0。

代码：

```
1  int maxSum2(vector<int>& nums, int k) {
2      int n = nums.size();
3      if (n <= k) return 0;
4      int l = 0, r = 0;
5      long long sum = 0;
6      int ans = INT_MIN;
7      for (int i = 0; i < n; i++) {
8          while (l < r && nums[window[r - 1]] >= nums[i]) r--;
9          window[r++] = i;
10         sum += nums[i];
11         if (i >= k) {
12             ans = max(ans, (int)(sum - nums[window[l]]));
13             if (window[l] == i - k) l++;
14             sum -= nums[i - k];
15         }
16     }
17     return ans;
18 }
```

class072 最长递增子序列问题与拓展

核心知识

最长递增子序列 (LIS) 最优解 — $O(n \log n)$

核心数据结构：ends 数组

- ends[i] = 目前发现的所有长度为 i+1 的递增子序列中，**最小的结尾值**
- ends 数组**严格升序**（证明：若长度为6的LIS结尾为21，则其前一个数必<21，所以长度为5的LIS最小结尾不可能≥21）

更新流程：

- 遍历每个数 x
- 在 ends[0..len-1] 中二分找 $\geq x$ 的最左位置
- 找到了** → 更新该位置为 x（更小的结尾更优）
- 没找到** → 扩充有效区 ends[len++] = x（冲到了新长度）

关键二分 (class072 核心)：

问题	二分策略	函数
严格递增子序列	<code>>= num</code> 最左	<code>bs1</code>
不下降子序列	<code>> num</code> 最左	<code>bs2</code>
不上升子序列（右→左）	<code>< num</code> 最左	<code>bs1</code> (P8776)

重要澄清： `ends` 数组的最终状态**不代表**任何一个实际的LIS，它只是每个长度的最小结尾信息的集合。

题目

01 最长递增子序列 — LeetCode 300

思路演进：

- $O(n^2)$ DP： `dp[i]` = 以 `i` 结尾的LIS长度，向左扫描比它小的数
- $O(n \log n)$ ： `ends` 数组 + 二分加速

核心点： `ends` 的严格升序性保证了二分正确性。最终答案 = `len`（有效区长度），不需要单独维护 `dp` 数组。

坑点： `ends` 与 `std::ends` 冲突，C++ 中必须改名（如 `edp`）。

02 俄罗斯套娃信封 — LeetCode 354

核心技巧——排序策略：

- 宽度从小到大
- 宽度相同时，高度从大到小

为什么高度要降序？ 防止相同宽度的信封被套在一起。如果高度升序，如 `(2,4)` `(2,9)`，在高度数组 `[4,9]` 中会形成长度为2的LIS，但实际上宽度相同的信封不能互套。

正确性： 排序后求高度LIS，只会找到「宽度更小 且 高度更小」的信封组合。

坑点： 排序比较器要正确处理 `a[0]==b[0]` 时高度降序，否则同宽信封会被误套。

03 使数组K递增 — LeetCode 2111

核心洞察——分组独立：

- 按 `i % k` 分成 `k` 组，每组之间无关
- `arr[i-k] <= arr[i]` 等价于每组内不下降

解法： 每组长度 - 该组最长不下降子序列长度 = 该组最少修改次数，各组累加。

坑点： 是**不下降**不是严格递增，二分要用 `> num` 最左（`bs2`），不要用错。

04 最长数对链 — LeetCode 646

解法一：LIS + 分离技巧

- 按开始位置排序
- 查：用 `pair[0]`（开头）在 `ends` 中查 $\geq$ 最左
- 改：用 `pair[1]`（结尾）更新 `ends`，且只有真的更小才改

解法二：贪心

- 按结尾位置排序
- 每次选能接上的（开头 > 上一个结尾）就接上
- 更简洁， $O(n \log n)$ ，最优

核心点（分离技巧）： 查的东西（开头） $\neq$ 入 `ends` 的东西（结尾），且更新有决策（`min(ends[find], pair[1])`）。

坑点： LIS解法中 `ends[find] = min(ends[find], pair[1])`，必须取 `min`，不是直接覆盖。

05 修改K个连续值 — Luogu P8776

核心洞察——紧贴性： 修改的 K 个连续值，一定紧挨着某个被选入最终子序列的数，不需要中间有空隙（因为空出来的元素反正不要， K 个紧贴着刷同样不下降）。

枚举框架：

```
1 对每个分界点 j（第一个不被修改的位置）：
2   ans = max(ans, left + K + rightArr[j])
3   其中 left = 左侧不下降子序列长度（结尾  $\leq arr[j]$ ）
4   rightArr[j] = 必须以 arr[j] 开头的右侧不下降子序列长度
```

分离技巧（滑动窗口）：

- 查 `ends`：当前分界值 `arr[j]`（决定左侧能取多少）
- 入 `ends`：滑出窗口的 `arr[i]`（新暴露的左边界）

`rightArr` 求解——反向最长不上升：

- 从右往左遍历
- `rightArr[i]` = 以 `arr[i]` 开头的最长不下降子序列
- = 从右往左看，以 `arr[i]` 结尾的最长不上升子序列
- 二分：< num 最左 (bs1)

坑点：

1. 最后一种情况： K 个在末尾，右侧为空，`ans = max(ans, len + K)`
2. `ends` 在 `computeRight()` 和主循环中复用，注意两次使用的二分策略不同 (bs1 vs bs2)
3. `rightArr` 的含义要理解清楚：一定以该位置开头的后缀LDS长度

技巧总结

技巧	出现题目
ends 数组 + 二分加速	01, 02, 03
排序策略消除一维	02 (宽度升+高度降)
分组独立	03 (按 mod k)
查改分离	04 (查开头改结尾) , 05 (查分界入滑出)
反向遍历等价转换	05 (右→左 不上升 = 左→右 不下降)
贪心替代DP	04 (按结尾排序贪心选)

老师的话

掌握更多算法和数据结构 = 更多被启发的机会。与其用有限的工具绞尽脑汁绕弯子，不如扩展武器库，让难题变成模板题。这就是「重剑无锋，大巧不工」的暴力美学。

class073 背包DP — 01背包、有依赖的背包

B站视频: [算法讲解073](#)
前置知识: class067、class068 (二维DP及空间压缩)

01 01背包模板 — 洛谷 P1048

题面: 背包容量t, n个物品各有体积w[i]和价值v[i], 每件最多选一次, 求最大价值。

核心思路: `dp[i][j]` = 前i个物品容量不超过j的最大价值。转移: ①不要i→ `dp[i-1][j]`; ②要i→ `dp[i-1][j-w[i]]+v[i]` (需 `j>=w[i]`)。空间压缩: 一维数组**从右往左**更新, `j<w[i]` 时天然继承上一行不更新。

坑点: 从右往左是为了保证读到的左侧值还是上一行的 (未被覆盖)。

代码:

```
1 // 空间压缩版
2 int dp[MAXT];
3 for (int i = 1; i <= n; i++) {
4     for (int j = t; j >= cost[i]; j--) {
5         dp[j] = max(dp[j], dp[j - cost[i]] + value[i]);
6     }
7 }
8 return dp[t];
```

02 夏季特惠 — LeetCode 未公开

题面: 预算X元, 每个商品有原价、现价、快乐值。总优惠金额≥总超过金额即可购买。求最大快乐值。

核心思路：转化： $\text{delta} = (\text{原价} - \text{现价}) - \text{现价} = \text{优惠} - \text{现价}$ 。若 $\text{delta} \geq 0 \rightarrow$ 必买，预算加 delta ，快乐值直接累加；若 $\text{delta} < 0 \rightarrow$ 待定商品，真正花费 $= -\text{delta}$ 。最后在待定商品上跑01背包。

坑点： $\text{delta} \geq 0$ 的商品不能放进背包——它们不消耗预算反而增加预算，必须直接处理。

代码：

```
1 while (m--) {
2     int pre, cur, happy; cin >> pre >> cur >> happy;
3     int well = (pre - cur) - cur; // 优惠 - 现价
4     if (well >= 0) {
5         x += well; ans += happy; // 必买
6     } else {
7         cost[++idx] = -well; // 真正花费
8         val[idx] = happy;
9     }
10 }
11 // 然后对cost[1..idx]跑01背包
12 for (int i = 1; i <= idx; i++)
13     for (int j = x; j >= cost[i]; j--)
14         dp[j] = max(dp[j], dp[j - cost[i]] + val[i]);
15 return ans + dp[x];
```

03 目标和 — LeetCode 494

题面：非负数组，每个数前添加+或-后串联成表达式，求结果等于target的表达式数量。

核心思路：设A=正号集合累加和，B=负号集合累加和，则 $\text{sumA} - \text{sumB} = \text{target}$ 且 $\text{sumA} + \text{sumB} = \text{sum}$ ，解得 $\text{sumA} = (\text{target} + \text{sum}) / 2$ 。问题变成：**有多少子序列累加和恰好为 $(\text{sum} + \text{target}) / 2$** 。01背包求方案数： $\text{dp}[j] += \text{dp}[j - \text{nums}[i]]$ （从右往左）。剪枝： $\text{target} > \text{sum}$ 或 $(\text{target} \& 1) \neq (\text{sum} \& 1) \rightarrow$ 返回0。

额外技巧—负数下标平移：暴力尝试版sum可能为负，把所有sum平移到 $[0, 2 * \text{sum}]$ 区间。

坑点： $(\text{target} + \text{sum})$ 必须是偶数（奇偶性不同则无解）；空集方案数为1（ $\text{dp}[0] = 1$ ）。

代码：

```
1 int findTargetSumWays(vector<int>& nums, int target) {
2     int sum = 0; for (int x : nums) sum += x;
3     if (sum < target || ((target & 1) != (sum & 1))) return 0;
4     return subsets(nums, (target + sum) / 2);
5 }
6 int subsets(vector<int>& nums, int t) {
7     if (t < 0) return 0;
8     vector<int> dp(t + 1);
9     dp[0] = 1;
10    for (int num : nums)
11        for (int j = t; j >= num; j--)
12            dp[j] += dp[j - num];
13    return dp[t];
14 }
```

04 最后一块石头的重量 II — LeetCode 1049

题面：一堆石头，每次取两块对撞：重量相同→全碎；不同→剩差值。最后最多剩一块，求最小可能重量。

核心思路：转化：将石头分成A、B两组对撞，最终重量=|sumA-sumB|。让两组之差尽量小=在数组中选若干个数使累加和尽量接近 sum/2。01背包：dp[j]=容量不超过j能凑出的最大累加和。答案= sum - 2*dp[sum/2]。

坑点：是尽量接近但不超过 sum/2，不是恰好等于。

代码：

```
1 int lastStoneweightII(vector<int>& stones) {
2     int sum = 0; for (int s : stones) sum += s;
3     int t = sum / 2;
4     vector<int> dp(t + 1);
5     for (int s : stones)
6         for (int j = t; j >= s; j--)
7             dp[j] = max(dp[j], dp[j - s] + s);
8     return sum - 2 * dp[t];
9 }
```

05 有依赖的背包 — 洛谷 P1064

题面：物品分主件和附件，每个主件最多2个附件。必须先买主件才能买其附件。给定总钱数，求最大收益（价格×重要度）。

核心思路：不是每个物品单独展开，而是以主件为单位做可能性展开。每个主件组有最多5种方案：①不要主件；②只要主件；③主件+附件1；④主件+附件2；⑤主件+附件1+附件2。辅数组：king[i] 标记是否主件、fans[i] 附件数、follows[i][2] 附件编号。只有主件才更新DP表，副件跟随主件展开。空间压缩：一维数组从右往左。

坑点：记得用变量p记录上一个展开的主件行号，因为上一行可能是跳过的副件行。

代码：

```
1 // 空间压缩版核心
2 for (int i = 1; i <= m; i++) {
3     if (!king[i]) continue; // 副件跳过
4     for (int j = n; j >= cost[i]; j--) {
5         // 方案②：只要主件
6         dp[j] = max(dp[j], dp[j - cost[i]] + val[i]);
7         int f1 = fans[i] >= 1 ? follows[i][0] : -1;
8         int f2 = fans[i] >= 2 ? follows[i][1] : -1;
9         // 方案③：主件+附件1
10        if (f1 != -1 && j - cost[i] - cost[f1] >= 0)
11            dp[j] = max(dp[j], dp[j - cost[i] - cost[f1]] + val[i] + val[f1]);
12        // 方案④：主件+附件2
13        if (f2 != -1 && j - cost[i] - cost[f2] >= 0)
14            dp[j] = max(dp[j], dp[j - cost[i] - cost[f2]] + val[i] + val[f2]);
15        // 方案⑤：全要
16        if (f1 != -1 && f2 != -1 && j - cost[i] - cost[f1] - cost[f2] >= 0)
17            dp[j] = max(dp[j], dp[j - cost[i] - cost[f1] - cost[f2]]
18                + val[i] + val[f1] + val[f2]);
19    }
```

```
19     }
20 }
21 return dp[n];
```

class074 背包DP — 分组背包、完全背包

B站视频: [算法讲解074](#)

前置知识: class067、class068、class073

01 分组背包模板

题面: 物品分若干组，每组最多选一件。每个物品有体积和价值，背包容量 t ，求最大价值。

核心思路: $dp[i][j]$ = 1~ i 组容量不超过 j 的最大价值。转移: ①不选 i 组 $\rightarrow dp[i-1][j]$; ②选 i 组内某件 $k \rightarrow dp[i-1][j - cost[k]] + val[k]$ 。预处理: 按组号排序物品。空间压缩: 一维数组**从右往左**更新。

坑点: $dp[j]$ 需先继承上一行 (天然继承不用写)，然后组内每件物品都尝试，注意取 $\max$ 而非直接赋值。

代码:

```
1 // 空间压缩版
2 for (int s = 1, e = 2; s <= n; s++) {
3     while (e <= n && arr[e][2] == arr[s][2]) e++;
4     // 当前组物品从 s 到 e-1
5     for (int j = t; j >= 0; j--) {
6         for (int k = s; k < e; k++) {
7             if (j >= arr[k][0])
8                 dp[j] = max(dp[j], dp[j - arr[k][0]] + arr[k][1]);
9         }
10    }
11    s = e; e++;
12 }
13 return dp[t];
```

02 栈中取K个硬币的最大价值 — LeetCode 2218

题面: 桌上 n 叠硬币 (每叠是一个栈，只能从顶拿)，每次操作取一个栈顶硬币。必须恰好 k 次操作，求最大总价值。

核心思路: 转化为分组背包: 每叠是一组，组内**方案**=取前 i 个硬币 ($i=0,1,2,\dots$)，消耗= i 次操作，收益=前缀和。每组只能选一个方案 (一种物品)。 $dp[i][j]$ = 1~ i 组恰好 j 次操作的最大价值。背包容量= k 。

坑点: 前缀和只需算到 $\min(\text{栈高度}, k)$; dp 的第二维是恰好 j 次而非不超过 j 次。

代码:

```
1 int maxValueOfCoins(vector<vector<int>>& piles, int m) {
2     int n = piles.size();
3     vector<vector<int>> dp(n + 1, vector<int>(m + 1, 0));
4     for (int i = 1; i <= n; i++) {
5         auto& pile = piles[i - 1];
```



```

6         int t = min((int)pile.size(), m);
7         vector<int> presum(t + 1);
8         for (int j = 1; j <= t; j++)
9             presum[j] = presum[j - 1] + pile[j - 1];
10        for (int j = 0; j <= m; j++) {
11            dp[i][j] = dp[i - 1][j]; // 不选这组
12            for (int k = 1; k <= min(j, t); k++)
13                dp[i][j] = max(dp[i][j], dp[i - 1][j - k] + presum[k]);
14        }
15    }
16    return dp[n][m];
17 }

```

03 完全背包模板 — 洛谷 P1616

题面：背包容量 t ， n 种物品各有体积和价值，每种可以选无限次，求最大价值。

核心思路：与01背包的唯一区别： $dp[i][j] = \max(dp[i-1][j], dp[i][j-w] + v)$ （停留在第 i 行而非 $i-1$ 行）。空间压缩：一维数组从左往右更新。

坑点：此题 t 可达 10^7 ，必须空间压缩，开二维表会爆内存。循环从 $j = cost[i]$ 开始而非 $j = t$ 。

代码：

```

1 vector<long long> dp(t + 1);
2 for (int i = 1; i <= n; i++)
3     for (int j = cost[i]; j <= t; j++) // 从左往右
4         dp[j] = max(dp[j], dp[j - cost[i]] + val[i]);
5 return dp[t];

```

04 正则表达式匹配 — LeetCode 10

题面：给字符串 s 和模式 p 。 $.$ 匹配任意单字符， $*$ 让前一个字符出现任意次（含0次）。判断 p 能否完全匹配 s 。

核心思路：递归→DP。 $f(i, j)$: $s[i..]$ 能否被 $p[j..]$ 匹配。分类：① $j+1$ 不是 $*$ → $s[i]$ 和 $p[j]$ 对上且 $f(i+1, j+1)$ ；② $j+1$ 是 $*$ →要么用0次（跳过 x^* ， $f(i, j+2)$ ），要么 $s[i]$ 和 $p[j]$ 对上且用1次（ $f(i+1, j)$ ，完全背包思想停在 j ）。DP: $dp[i][j]$ 依赖右下+同行跳两格→从下往上、从右往左填。

坑点： s 为空时 p 可能配得上（如 $a^*b^*c^*$ 全消为0个），base case要处理。

代码：

```

1 class Solution {
2 public:
3     // 暴力递归
4     // s[i....]能不能被p[j....]完全匹配出来
5     // p[j]这个字符，一定不是'*'
6     bool f1(const string& s, const string& p, int i, int j) {
7         if (i == (int)s.length()) {
8             // s没了
9             if (j == (int)p.length()) {

```

```

10         // 如果p也没了, 返回true
11         return true;
12     } else {
13         // p还剩下一些后缀
14         // 如果p[j+1]是*, 那么p[j..j+1]可以消掉, 然后看看p[j+2....]是不是都能消掉
15         return j + 1 < (int)p.length() && p[j + 1] == '*' && f1(s, p, i, j +
2);
16     }
17 } else if (j == (int)p.length()) {
18     // s有后缀, p没后缀了
19     return false;
20 } else {
21     // s有后缀, p有后缀
22     if (j + 1 == (int)p.length() || p[j + 1] != '*') {
23         // 如果p[j+1]不是*, 那么当前的字符必须能匹配
24         // 同时, 后续也必须匹配上
25         return (s[i] == p[j] || p[j] == '.') && f1(s, p, i + 1, j + 1);
26     } else {
27         // 如果p[j+1]是* -- 完全背包思想!
28         // 选择1: 当前p[j..j+1]是x*, 不让他搞定s[i], 继续 f(i, j+2)
29         bool p1 = f1(s, p, i, j + 2);
30         // 选择2: 当前p[j..j+1]是x*, 如果可以搞定s[i], 继续 f(i+1, j)
31         // 如果可以搞定s[i] : (s[i] == p[j] || p[j] == '.')
32         bool p2 = (s[i] == p[j] || p[j] == '.') && f1(s, p, i + 1, j);
33         return p1 || p2;
34     }
35 }
36 }
37
38 bool isMatch1(string s, string p) {
39     return f1(s, p, 0, 0);
40 }
41
42 // 记忆化搜索
43 // dp[i][j] == 0 没算过, 1 算过true, 2 算过false
44 bool isMatch2(string s, string p) {
45     int n = s.length(), m = p.length();
46     vector<vector<int>> dp(n + 1, vector<int>(m + 1, 0));
47     return f2(s, p, 0, 0, dp);
48 }
49
50 bool f2(const string& s, const string& p, int i, int j, vector<vector<int>>& dp) {
51     if (dp[i][j] != 0) return dp[i][j] == 1;
52     bool ans;
53     if (i == (int)s.length()) {
54         if (j == (int)p.length()) {
55             ans = true;
56         } else {
57             ans = j + 1 < (int)p.length() && p[j + 1] == '*' && f2(s, p, i, j + 2,
dp);
58         }
59     } else if (j == (int)p.length()) {
60         ans = false;

```

```

61         } else {
62             if (j + 1 == (int)p.length() || p[j + 1] != '*') {
63                 ans = (s[i] == p[j] || p[j] == '.') && f2(s, p, i + 1, j + 1, dp);
64             } else {
65                 ans = f2(s, p, i, j + 2, dp)
66                     || ((s[i] == p[j] || p[j] == '.') && f2(s, p, i + 1, j, dp));
67             }
68         }
69         dp[i][j] = ans ? 1 : 2;
70         return ans;
71     }
72
73     // 严格位置依赖的动态规划
74     bool isMatch(string s, string p) {
75         int n = s.length(), m = p.length();
76         vector<vector<bool>> dp(n + 1, vector<bool>(m + 1, false));
77         dp[n][m] = true;
78         // 处理s为空时p的后缀：只有 x*x*... 模式才能消掉
79         for (int j = m - 1; j >= 0; j--) {
80             dp[n][j] = j + 1 < m && p[j + 1] == '*' && dp[n][j + 2];
81         }
82         for (int i = n - 1; i >= 0; i--) {
83             for (int j = m - 1; j >= 0; j--) {
84                 if (j + 1 == m || p[j + 1] != '*') {
85                     dp[i][j] = (s[i] == p[j] || p[j] == '.') && dp[i + 1][j + 1];
86                 } else {
87                     dp[i][j] = dp[i][j + 2] || ((s[i] == p[j] || p[j] == '.') && dp[i +
1][j]);
88                 }
89             }
90         }
91         return dp[0][0];
92     }
93 };

```

05 通配符匹配 — LeetCode 44

题面：类似题目4但?匹配任意单字符，*匹配任意字符串（不依赖前字符）。比题目4简单。

核心思路： $f(i, j)$: ① $p[j]$ 不是 * $\rightarrow$ 当前对上且 $f(i+1, j+1)$; ② $p[j]$ 是 * $\rightarrow$ 不匹配 $s[i]$ (跳过 *, $f(i, j+1)$) 或匹配 $s[i]$ (停在 *, $f(i+1, j)$)。完全背包思想一致。

代码：略（与题目4结构类似，更简单）。

06 购买足量干草的最小花费

题面： n 个公司各有一种产品，花费 $c[i]$ 得 $v[i]$ 千克草，可无限买。必须凑 $\geq H$ 千克，求最小花费。

核心思路：定义翻转： $dp[i][j]$ = 前 i 公司严格 j 千克的最小花费（不再求最大价值）。关键扩充：严格等于 H 可能无解 $\rightarrow$ 列扩充到 $H + \max(v[i])$ （覆盖最近整数倍），最后枚举 $j \in [H, H + \max v]$ 取最小 $dp[n][j]$ 。转移： $dp[i][j] = \min(dp[i-1][j], dp[i][j-v[i]]+c[i])$ 。

坑点：无效解用 `INT_MAX` 表示，转移前要检查 `dp[i][j-v[i]]` 不是无效解。

代码：

```
1 // 空间压缩版
2 vector<int> dp(m + 1, INT_MAX); // m = H + maxV
3 dp[0] = 0;
4 for (int i = 1; i <= n; i++)
5     for (int j = val[i]; j <= m; j++)
6         if (dp[j - val[i]] != INT_MAX)
7             dp[j] = min(dp[j], dp[j - val[i]] + cost[i]);
8 int ans = INT_MAX;
9 for (int j = H; j <= m; j++) ans = min(ans, dp[j]);
10 return ans;
```

class075 背包DP — 多重背包、混合背包

B站视频：[算法讲解075](#)

前置知识：class054 单调队列、class073 01背包、class074 完全背包

01 多重背包 — 朴素枚举（会超时）

题面：n种物品，每种有体积w、价值v、数量cnt。背包容量t，求最大价值。

核心思路： $dp[i][j] = \max_{k=0..cnt} (dp[i-1][j-k*w] + k*v)$ 。朴素枚举会超时 ($O(n \cdot t \cdot avgCnt)$)。

坑点：需同时限制 $k \leq cnt$ 且 $k*w \leq j$ 。

代码：

```
1 // 朴素版，大cnt会超时
2 for (int i = 1; i <= n; i++) {
3     for (int j = 0; j <= t; j++) {
4         dp[i][j] = dp[i-1][j];
5         for (int k = 1; k <= cnt[i] && k * w[i] <= j; k++) {
6             dp[i][j] = max(dp[i][j], dp[i-1][j - k*w[i]] + k*v[i]);
7         }
8     }
9 }
```

02 多重背包 — 二进制分组优化（最常用）

题面：同01，但必须优化。

核心思路：把cnt个物品拆成 $\log(cnt)$ 个"包裹"，1,2,4,8,...倍捆绑+最后小尾巴。每个包裹是要么全拿、要么全不拿的01背包物品。原理：二进制组合能覆盖 $[0, cnt]$ 所有数字。最后对包裹跑01背包($O(\text{包裹数} \cdot t)$)。

坑点：最后剩的小尾巴不能漏；有些数量可能被重复表示，但不影响正确性。

代码：

```

1  #include<bits/stdc++.h>
2  using namespace std;
3
4  const int maxn = 1005;
5  const int maxm = 40005;
6
7  int n,t,idx;
8  int val[maxn];
9  int wei[maxn];
10 int dp[maxm];
11
12 int main(){
13     ios::sync_with_stdio(false);
14     cin.tie(nullptr);
15     while(cin>>n>>t){
16         idx = 0;
17         for (int i = 1, v, w, c; i <= n; ++i){
18             cin >> v >> w >> c;
19             for (int k = 1; k <= c;k<=1){
20                 val[++idx] = k * v;
21                 wei[idx] = k * w;
22                 c -= k;
23             }
24             if(c>0){
25                 val[++idx] = c * v;
26                 wei[idx] = c * w;
27             }
28         }
29         for (int i = 1; i <= idx;++i){
30             for (int j = t; j >= wei[i]; --j){
31                 dp[j] = max(dp[j], dp[j - wei[i]] + val[i]);
32             }
33         }
34         cout << dp[t] << '\n';
35     }
36     return 0;
37 }

```

03 观赏樱花

题面：给定开始/结束时间，求欣赏时长。 n 种樱花，每种消耗分钟数、获得美学价值、可看次数（0=无限）。求最大美学价值。

核心思路：无限次→补次数为1001（总容量上限），然后统一二进制分组+01背包。时间解析：小时差 $\times 60$ +分钟差，若开始分钟 $>$ 结束分钟则借位。

坑点：输入格式含冒号，需跳过。

代码：

```

1 // 关键转化
2 int cnt;
3 cin >> cost >> val >> cnt;
4 if (cnt == 0) cnt = 1001; // 无限次补足
5 // 然后二进制分组

```

04 多重背包 — 单调队列优化（最优复杂度）

题面：同01，求 $O(n \cdot t)$ 解。

核心思路：①**按余数分组：**重量 w ，按 $j \% w$ 分成 w 组，同组内格子只依赖同组($j, j-w, j-2w \dots$)；②**指标改写：**把 $dp[i-1][j]+k*v$ 改写为 $(j/w)*v + \max \text{ over窗口}(dp[i-1][G]-G/w*v)$ ，式中 G 是窗口内各位置。指标只与 G 有关，不随 j 变化，可进单调队列；③**滑动窗口：**每次进一个新指标（ G 变小方向），出一个过期指标（超 cnt 个）。

空间压缩版从右往左+提前建窗口。

坑点：理解关键是指标改写让值固定下来；窗口长度= $cnt+1$ （含0个的情况）。

代码：

```

1 // 单调队列优化（空间压缩版核心逻辑）
2 for (int i = 1; i <= n; i++) {
3     int w = cost[i], v = val[i], cnt = num[i];
4     for (int mod = 0; mod < min(t + 1, w); mod++) {
5         int l = 0, r = 0;
6         // 提前建窗口（先放cnt个，少放1个）
7         for (int j = t - mod, enter = 0; j >= 0 && enter < cnt; j -= w, enter++) {
8             while (l < r && value1(dp, j) >= value1(dp, window[r - 1])) r--;
9             window[r++] = j;
10        }
11        // 逐格求解
12        for (int j = t - mod, enter = 0; j >= 0; j -= w) {
13            // 新指标进入
14            int nextJ = j - (cnt + 1) * w;
15            if (nextJ >= 0) {
16                while (l < r && value1(dp, nextJ) >= value1(dp, window[r - 1])) r--;
17                window[r++] = nextJ;
18            }
19            // 求dp[j]
20            if (l < r) dp[j] = value1(dp, window[l]) + (j / w) * v;
21            // 当前j过期
22            if (l < r && window[l] == j) l++;
23        }
24    }
25 }
26 // value1: 指标值 = dp[g] - (g/w)*v

```

05 混合背包找零钱 — POJ 1742

题面： n 种货币，面值 $a[i]$ 和数量 $c[i]$ 。求 $[1..m]$ 中有多少金额能被凑出（不限制凑法，只要能凑出）。

核心思路：对每种货币分三种情况处理：① $cnt==1 \rightarrow 01$ 背包，从右往左；② $面值 \times cnt \geq M \rightarrow$ 完全背包（根本用不完），从左往右；③其他 $\rightarrow$ 多重背包窗口优化。由于是布尔类型，不需要单调队列pk指标，只需统计窗口内true的个数 >0 即可。每个余数组维护一个true计数，进窗口+1/出窗口-1。

坑点：空间卡得严，必须空间压缩；右左左方向取决于类型。

代码：

```
1 // 三种类型核心逻辑
2 for (int i = 1; i <= n; i++) {
3     if (cnt[i] == 1) {
4         // 01背包，从右往左
5         for (int j = m; j >= a[i]; j--)
6             if (dp[j - a[i]]) dp[j] = true;
7     } else if (a[i] * cnt[i] >= m) {
8         // 完全背包，从左往右
9         for (int j = a[i]; j <= m; j++)
10             if (dp[j - a[i]]) dp[j] = true;
11     } else {
12         // 多重背包窗口优化（布尔版）
13         for (int mod = 0; mod < a[i]; mod++) {
14             int trueCnt = 0;
15             // 初始窗口
16             for (int j = m - mod, k = 0; j >= 0 && k <= cnt[i]; j -= a[i], k++)
17                 if (dp[j]) trueCnt++;
18             // 滑动窗口
19             for (int j = m - mod, enter; j >= 0; j -= a[i]) {
20                 enter = j - (cnt[i] + 1) * a[i];
21                 if (enter >= 0 && dp[enter]) trueCnt++;
22                 if (trueCnt > 0) dp[j] = true;
23                 if (dp[j]) trueCnt--;
24             }
25         }
26     }
27 }
28 int ans = 0;
29 for (int j = 1; j <= m; j++) if (dp[j]) ans++;
30 return ans;
```

class076 区间DP（上）

B站视频：[算法讲解076](#)
前置知识：class067、class068（二维DP及空间压缩）

核心知识：区间DP两种展开方式

区间DP = 解决大范围 $[L, R]$ 的问题，拆成小范围再汇总。两种常见展开方式：

方式	特点	复杂度	题目
基于两侧端点	依赖 $L+1/R-1$ 等固定位置，无for枚举	$O(n^2)$	1, 2

方式	特点	复杂度	题目
基于范围划分点	枚举中间划分点 m ，左右分别求解	$O(n^3)$	3, 4, 5, 6

01 让字符串成为回文串的最少插入次数 — LeetCode 1312

题面： 给定字符串，可在任意位置插入任意字符，求使其成为回文串的最少插入次数。

核心思路： $f(l, r)$ = 让 $s[l..r]$ 变回文的最少插入数。分情况：

- $l == r \rightarrow 0$ (单字符天然回文)
- $l+1 == r \rightarrow s[l] == s[r] ? 0 : 1$
- $s[l] == s[r] \rightarrow f(l+1, r-1)$ (两端自我消化)
- $s[l] != s[r] \rightarrow \min(f(l+1, r), f(l, r-1)) + 1$ (搞定一端，加一个字符配它)

坑点： 无。

代码：

```

1  // 记忆化搜索
2  class Solution {
3  public:
4      int dp[505][505];
5
6      int f1(string& s, int l, int r){
7          if(dp[l][r] != -1)
8              return dp[l][r];
9          int ans = 0;
10         if(l == r){
11             ans = 0;
12         }else if(l+1 == r){
13             ans = s[l] == s[r] ? 0 : 1;
14         }else{
15             if(s[l] == s[r]){
16                 ans = f1(s, l + 1, r - 1);
17             }else{
18                 ans = min(f1(s, l + 1, r), f1(s, l, r - 1)) + 1;
19             }
20         }
21         dp[l][r] = ans;
22         return ans;
23     }
24
25     int minInsertions(string s) {
26         int n = s.length();
27         for (int i = 0; i < n; ++i){
28             for (int j = 0; j < n; ++j)
29                 dp[i][j] = -1;
30         }
31         return f1(s, 0, n - 1);
32     }

```



```

33 };
34
35 // 严格位置依赖的动态规划
36 class solution {
37 public:
38     int dp[505][505];
39
40     int minInsertions(string s) {
41         int n = s.length();
42         for (int i = 0; i < n; ++i)
43             dp[i][i] = 0;
44         for (int l = 0; l < n - 1; ++l)
45             dp[l][l + 1] = s[l] == s[l + 1] ? 0 : 1;
46         for (int l = n - 3; l >= 0; --l){
47             for (int r = l + 2; r < n; ++r){
48                 if(s[l]==s[r]){
49                     dp[l][r] = dp[l + 1][r - 1];
50                 }else{
51                     dp[l][r] = min(dp[l + 1][r], dp[l][r - 1]) + 1;
52                 }
53             }
54         }
55         return dp[0][n - 1];
56     }
57 };

```

02 预测赢家 — LeetCode 486

题面：非负数组，两人轮流从两端取数，都绝顶聪明（零和博弈）。玩家1先手，问玩家1能否获胜（得分 $\geq$ 玩家2）。

核心思路： $f(l, r)$ = 在 $[l..r]$ 上先手能得的最大分数。玩家1拿走 $nums[l]$ 后，玩家2会在 $[l+1..r]$ 上做对自己最有利的选择，给玩家1留下后续的**最小值**（因为零和博弈）：

```

1  p1 = nums[l] + min(f(l+2, r), f(l+1, r-1)) // 拿左边
2  p2 = nums[r] + min(f(l+1, r-1), f(l, r-2)) // 拿右边
3  return max(p1, p2)

```

答案： `sum =  $\sum$ nums`, `first = dp[0][n-1]`, `second = sum - first`, `return first >= second`。

坑点：每个玩家的得分决策都基于“给对手留下最差局面”。

代码：

```

1  class solution {
2  public:
3      int dp[25][25];
4
5      int f(vector<int>& nums, int l, int r){
6          if(dp[l][r] != -1)
7              return dp[l][r];
8          int ans;
9          if (l == r){

```

```

10         ans = nums[l];
11     }else if(l==r-1){
12         ans = nums[l] > nums[r] ? nums[l] : nums[r];
13     }else{
14         int p1 = nums[l] + min(f(nums, l + 2, r), f(nums, l + 1, r - 1));
15         int p2 = nums[r] + min(f(nums, l + 1, r - 1), f(nums, l, r - 2));
16         ans = max(p1, p2);
17     }
18     dp[l][r] = ans;
19     return ans;
20 }
21
22 bool predictTheWinner(vector<int>& nums) {
23     int sum = 0;
24     int n = nums.size();
25     for(int n:nums)
26         sum += n;
27     for (int i = 0; i < n;++i){
28         for (int j = 0; j < n;++j)
29             dp[i][j] = -1;
30     }
31     int f1 = f(nums, 0, n - 1);
32     int f2 = sum - f1;
33     return f1 >= f2;
34 }
35 };

```

03 多边形三角剖分的最低得分 — LeetCode 1039

题面：凸 n 边形的顶点有权值，划分为 $n-2$ 个三角形，每个三角形得分=三个顶点值乘积，总分为各三角形得分之和。求最低总分。

核心思路： $f(l, r)$ = 顶点 $l \dots r$ 围成的多边形的最低三角剖分得分。选中间点 m ($l < m < r$)，三角形 (l, m, r) 的得分 $= arr[l] * arr[m] * arr[r]$ ，左侧 $f(l, m)$ +右侧 $f(m, r)$ 递归。枚举所有 m 取 $\min$ 。

坑点： $l == r$ 或 $l+1 == r$ 时无法构成三角形，返回0。 $O(n^3)$ 。

代码：

```

1  class Solution {
2  public:
3      int dp[55][55];
4
5      int f(vector<int>& arr, int l, int r){
6          if(dp[l][r] != -1)
7              return dp[l][r];
8          int ans = INT_MAX;
9          if(l == r || l+1 == r){
10             ans = 0;
11         }else{
12             for (int m = l + 1; m < r; ++m){
13                 ans = min(ans, f(arr, l, m) + arr[l] * arr[m] * arr[r] + f(arr, m, r));

```

```

14         }
15     }
16     dp[l][r] = ans;
17     return ans;
18 }
19
20 int minScoreTriangulation(vector<int>& arr) {
21     int n = arr.size();
22     for (int i = 0; i < n; ++i){
23         for (int j = 0; j < n; ++j)
24             dp[i][j] = -1;
25     }
26     return f(arr, 0, n - 1);
27 }
28 };
29
30
31 class Solution {
32 public:
33     int dp[55][55];
34
35     int minScoreTriangulation(vector<int>& arr) {
36         int n = arr.size();
37         for (int i = 0; i < n; ++i)
38             dp[i][i] = 0;
39         for (int i = 0; i < n - 1; ++i)
40             dp[i][i + 1] = 0;
41         for (int l = n - 3; l >= 0; --l){
42             for (int r = l + 2; r < n; ++r){
43                 dp[l][r] = INT_MAX;
44                 for (int m = l + 1; m < r; ++m){
45                     dp[l][r] = min(dp[l][r], dp[l][m] + dp[m][r] + arr[l] * arr[m] *
arr[r]);
46                 }
47             }
48         }
49         return dp[0][n - 1];
50     }
51 };

```

04 切棍子的最小成本 — LeetCode 1547

题面：长 n 的木棍，需要在cuts点切开。每次切割成本=当前切割段长度，可自由决定切割顺序。求最小总成本。

核心思路：给cuts排序后补0和 n （方便算长度）。 $f(l, r)$ = 切完 $[l..r]$ 切点的最小成本。选第一个切点 k ，当前段长= $arr[r+1]-arr[l-1]$ 加上左右递归。枚举所有 k 取min。 $O(m^3)$ ， m 是切点数量。

坑点：需要在cuts前后补0和 n 来算长度。

代码：

```

1 class Solution {

```

```

2 public:
3     int dp[105][105];
4
5     int f(int* arr,int l,int r){
6         if(l>r)
7             return 0;
8         if(l==r)
9             return arr[r + 1] - arr[l - 1];
10        if(dp[l][r]!=-1)
11            return dp[l][r];
12        int ans = INT_MAX;
13        for (int k = l; k <= r;++k){
14            ans = min(ans, f(arr,l, k - 1) + f(arr,k + 1, r));
15        }
16        ans += arr[r + 1] - arr[l - 1];
17        dp[l][r] = ans;
18        return ans;
19    }
20
21    int minCost(int n, vector<int>& cuts) {
22        int m = cuts.size();
23        sort(cuts.begin(), cuts.end());
24        int arr[m + 2];
25        arr[0] = 0;
26        for (int i = 1; i <= m; ++i){
27            arr[i] = cuts[i - 1];
28        }
29        arr[m + 1] = n;
30        for (int i = 0; i <= m;++i){
31            for (int j = 0; j <= m;++j)
32                dp[i][j] = -1;
33        }
34        return f(arr, 1, m);
35    }
36 };
37
38 int minCost(int n, vector<int>& cuts) {
39     int m = cuts.size();
40     sort(cuts.begin(), cuts.end());
41     arr[0] = 0; arr[m + 1] = n;
42     for (int i = 1; i <= m; i++) arr[i] = cuts[i - 1];
43     memset(dp, 0, sizeof(dp));
44     for (int i = 1; i <= m; i++)
45         dp[i][i] = arr[i + 1] - arr[i - 1];
46     for (int l = m - 1; l >= 1; l--)
47         for (int r = l + 1; r <= m; r++) {
48             int nxt = INT_MAX;
49             for (int k = l; k <= r; k++)
50                 nxt = min(nxt, dp[l][k - 1] + dp[k + 1][r]);
51             dp[l][r] = arr[r + 1] - arr[l - 1] + nxt;
52         }
53     return dp[1][m];
54 }

```

05 戳气球 — LeetCode 312

题面： `nums[i]` 个气球，戳爆 `i` 得 `nums[i-1]*nums[i]*nums[i+1]` 分。边界外=1。求最大得分。

核心思路： 关键洞察：尝试每个气球"最后打爆"（而不是最先打爆！）。设潜台词：调用 `f(l,r)` 时保证 `arr[l-1]` 和 `arr[r+1]` 一定没爆。预处理：首尾补1。枚举最后打爆的气球 `k`：

```
1 | ans = arr[l-1]*arr[k]*arr[r+1] + f(l, k-1) + f(k+1, r)
```

最先打爆不行→因为B的右侧"最近的没爆气球"会随之前的打爆顺序变化，信息不够。

坑点： 必须"最后打爆"的尝试方式，配合"两端没爆"的潜台词。

代码：

```
1 | class Solution {
2 | public:
3 |     int dp[310][310];
4 |
5 |     int f(int *arr, int l, int r){
6 |         if(dp[l][r] != -1)
7 |             return dp[l][r];
8 |         int ans;
9 |         if(l == r)
10 |             ans = arr[l - 1] * arr[l] * arr[r + 1];
11 |         else{
12 |             ans = max(arr[l-1]*arr[l]*arr[r+1]+f(arr, l+1, r),
13 |                      arr[l-1]*arr[r]*arr[r+1]+f(arr, l, r-1));
14 |             for (int k = l + 1; k < r; ++k){
15 |                 ans = max(ans, arr[l - 1] * arr[k] * arr[r + 1] + f(arr, l, k - 1) +
16 |                           f(arr, k + 1, r));
17 |             }
18 |             dp[l][r] = ans;
19 |             return ans;
20 |         }
21 |
22 |     int maxCoins(vector<int>& nums) {
23 |         int n = nums.size();
24 |         int arr[n + 2];
25 |         arr[0] = 1;
26 |         arr[n + 1] = 1;
27 |         for (int i = 1; i <= n; ++i)
28 |             arr[i] = nums[i - 1];
29 |         for (int i = 1; i <= n; ++i){
30 |             for (int j = 1; j <= n; ++j)
31 |                 dp[i][j] = -1;
32 |         }
33 |         return f(arr, 1, n);
34 |     }
35 | };
36 |
```

```

37
38 int maxCoins(vector<int>& nums) {
39     int n = nums.size();
40     arr[0] = arr[n + 1] = 1;
41     for (int i = 0; i < n; i++) arr[i + 1] = nums[i];
42     memset(dp, 0, sizeof(dp));
43     for (int i = 1; i <= n; i++)
44         dp[i][i] = arr[i - 1] * arr[i] * arr[i + 1];
45     for (int l = n; l >= 1; l--)
46         for (int r = l + 1; r <= n; r++) {
47             int ans = max(arr[l-1]*arr[l]*arr[r+1] + dp[l+1][r],
48                           arr[l-1]*arr[r]*arr[r+1] + dp[l][r-1]);
49             for (int k = l + 1; k < r; k++)
50                 ans = max(ans, arr[l-1]*arr[k]*arr[r+1]
51                           + dp[l][k-1] + dp[k+1][r]);
52             dp[l][r] = ans;
53         }
54     return dp[1][n];
55 }

```

06 布尔运算 — LeetCode 面试题 08.14

题面：字符串 0/1 与 &|^ 交替（范式：偶数下标数字，奇数下标逻辑符）。可加括号改变运算顺序。返回得到指定 result 的方法数。

核心思路： f(l,r) 返回 int[2]：[false 方法数, true 方法数]。枚举每个逻辑符 k 最后执行（k=l+1, l+3, ..., 步长2），左侧 f(l,k-1)、右侧 f(k+1,r)。根据 s[k] 的类型（&|^）组合左右 true/false 的方法数累加。

坑点：内存化搜索版本就够了；范式要求 l 和 r 位置必须是数字；逻辑符位置一定是奇数下标，枚举步长为2。

代码：

```

1  int* f(string& s, int l, int r) {
2      if (has[l][r]) return dp[l][r];
3      int fcnt = 0, tcnt = 0;
4      if (l == r) {
5          fcnt = s[l] == '0' ? 1 : 0;
6          tcnt = s[l] == '1' ? 1 : 0;
7      } else {
8          for (int k = l + 1; k < r; k += 2) {
9              int *tmp1 = f(s, l, k - 1), *tmp2 = f(s, k + 1, r);
10             int a = tmp1[0], b = tmp1[1], c = tmp2[0], d = tmp2[1];
11             if (s[k] == '&') { fcnt += a*c+a*d+b*c; tcnt += b*d; }
12             else if (s[k] == '|') { fcnt += a*c; tcnt += a*d+b*c+b*d; }
13             else { fcnt += a*c+b*d; tcnt += a*d+b*c; }
14         }
15     }
16     dp[l][r][0] = fcnt; dp[l][r][1] = tcnt; has[l][r] = true;
17     return dp[l][r];
18 }

```

class077 区间DP（下）

B站视频: [算法讲解077](#)

前置知识: class076 (区间DP上)

01 完成配对需要的最少字符数量（括号配对）

题面：字符串由 `[`、`]`、`(`、`)` 组成，最少插入多少个括号使所有括号正确配对。

核心思路： $f(l, r)$ = 让 $s[l..r]$ 正确配对的最少插入数。两种可能性：

- 可能性1: $s[l]$ 与 $s[r]$ 本就配对 $\rightarrow f(l+1, r-1)$
- 可能性2: 枚举划分点 m ，左右各自配对 $\rightarrow f(l, m) + f(m+1, r)$

取min。base case: $l==r \rightarrow 1$, $l+1==r \rightarrow$ 配对0/不配对2。

坑点：可能性1和可能性2都要考虑，不能只看两端配对。

代码：

```
1  const int maxn = 210;
2  int dp[maxn][maxn];
3  string s;
4
5  int f(int l, int r){
6      if(l==r)
7          return 1;
8      if(l+1==r){
9          if((s[l]=='(' && s[r]==')') || (s[l]=='[' && s[r]==']')){
10             return 0;
11          }else{
12             return 2;
13          }
14      }
15      if(dp[l][r]!=-1)
16          return dp[l][r];
17
18      int p1 = INT_MAX;
19      if ((s[l] == '(' && s[r] == ')') || (s[l] == '[' && s[r] == ']')){
20          p1 = f(l + 1, r - 1);
21      }
22      int p2 = INT_MAX;
23      for (int k = l; k < r; ++k){
24          p2 = min(p2, f(l, k) + f(k + 1, r));
25      }
26      int ans = min(p1, p2);
27      dp[l][r] = ans;
28      return ans;
29  }
30
31  int main(){
32      cin >> s;
```

```

33     int n = s.length();
34     for (int i = 0; i < n; ++i){
35         for (int j = 0; j < n; ++j)
36             dp[i][j] = -1;
37     }
38     cout << f(0, n - 1) << '\n';
39     return 0;
40 }

```

02 涂色（奇怪打印机） — LeetCode 664

题面： 每次可以把一段连续区间涂成同一颜色（后涂覆盖先涂），求涂出目标串的最少次数。

核心思路： $dp[l][r]$ = 涂出 $s[l..r]$ 的最少次数。

- 若 $s[l] == s[r] \rightarrow dp[l][r] = dp[l][r-1]$ （因为涂 $[l..r]$ 时顺带把最右涂了）
- 否则枚举划分点 $m \rightarrow dp[l][r] = \min(dp[l][m] + dp[m+1][r])$

坑点： $s[l] == s[r]$ 时 $dp[l][r-1]$ 和 $dp[l+1][r]$ 等价。

代码：

```

1  class Solution {
2  public:
3      int dp[105][105];
4
5      int f(string& s, int l, int r){
6          if(dp[l][r] != -1)
7              return dp[l][r];
8          int ans = INT_MAX;
9          if(l == r){
10             ans = 1;
11         }else if(l+1 == r){
12             ans = s[l] == s[r] ? 1 : 2;
13         }else{
14             if(s[l] == s[r]){
15                 ans = f(s, l, r - 1);
16             }else{
17                 for (int m = l; m < r; ++m){
18                     ans = min(ans, f(s, l, m) + f(s, m + 1, r));
19                 }
20             }
21         }
22         dp[l][r] = ans;
23         return ans;
24     }
25
26     int strangePrinter(string s) {
27         int n = s.length();
28         for (int i = 0; i < n; ++i){
29             for (int j = 0; j < n; ++j){
30                 dp[i][j] = -1;

```



```

31     }
32 }
33 return f(s, 0, n - 1);
34 }
35 };
36
37
38 int strangePrinter(string str) {
39     int n = str.length();
40     dp[n-1][n-1] = 1;
41     for (int i = 0; i < n-1; i++) {
42         dp[i][i] = 1;
43         dp[i][i+1] = str[i]==str[i+1] ? 1 : 2;
44     }
45     for (int l = n-3; l >= 0; l--)
46         for (int r = l+2; r < n; r++) {
47             if (str[l] == str[r])
48                 dp[l][r] = dp[l][r-1];
49             else {
50                 int ans = INT_MAX;
51                 for (int m = l; m < r; m++)
52                     ans = min(ans, dp[l][m] + dp[m+1][r]);
53                 dp[l][r] = ans;
54             }
55         }
56     return dp[0][n-1];
57 }

```

03 合唱队 — 洛谷 P3205

题面： n 个人按编号排列，有一个理想队形（给定数组）。每次从当前队伍的最左或最右选一个人加入新队列的末尾。求有多少种出队方式能得到理想队形，结果对 19650827 取模。

推导状态转移方程

从左边进来肯定前 1 个人比他高，前 1 个人有 2 种情况，要么在 $i+1$ 号位置，要么在 j 号位置。

同理，

从右边进来肯定前 1 个人比他矮，前 1 个人有 2 种情况，要么在 $j-1$ 号位置，要么在 i 号位置。

那么状态转移方程就出来了：

```

1  if(a[i]<a[i+1])f[i][j][0]+=f[i+1][j][0];
2  if(a[i]<a[j])f[i][j][0]+=f[i+1][j][1];
3  if(a[j]>a[i])f[i][j][1]+=f[i][j-1][0];
4  if(a[j]>a[j-1])f[i][j][1]+=f[i][j-1][1];
5  f[i][j][0]%=19650827;
6  f[i][j][1]%=19650827;

```

坑点： 第三维记录"最后加入的是左端还是右端"，从小区间往大区间推。

代码：

```

1  int compute() {
2      memset(dp3d, 0, sizeof(dp3d));
3      for (int i = 1; i < n; i++)
4          if (nums[i] < nums[i+1])
5              dp3d[i][i+1][0] = dp3d[i][i+1][1] = 1;
6      for (int l = n-2; l >= 1; l--)
7          for (int r = l+2; r <= n; r++) {
8              if (nums[l] < nums[l+1])
9                  dp3d[l][r][0] = (dp3d[l][r][0] + dp3d[l+1][r][0]) % MOD;
10             if (nums[l] < nums[r])
11                 dp3d[l][r][0] = (dp3d[l][r][0] + dp3d[l+1][r][1]) % MOD;
12             if (nums[r] > nums[l])
13                 dp3d[l][r][1] = (dp3d[l][r][1] + dp3d[l][r-1][0]) % MOD;
14             if (nums[r] > nums[r-1])
15                 dp3d[l][r][1] = (dp3d[l][r][1] + dp3d[l][r-1][1]) % MOD;
16         }
17     return (dp3d[1][n][0] + dp3d[1][n][1]) % MOD;
18 }

```

04 移除盒子 — LeetCode 546

题面： 盒子有颜色，每轮移除k个连续的相同颜色盒子得 $k*k$ 分。求最大总分。

核心思路： 三维dp。 $f(l, r, k)$ = $[l..r]$ 范围上消除，前面有k个与 $boxes[l]$ 同色的连续盒子。先合并两端同色的（s 指针跳过）， $cnt=k+s-l+1$ 。两种选择：①前缀直接消： $cnt*cnt+f(s+1, r, 0)$ ；②前缀跟后面的同色盒子一起消：枚举 m 满足 $boxes[l]==boxes[m]$ 且 $boxes[m-1]!=boxes[m]$ （剪枝）， $f(s+1, m-1, 0)+f(m, r, cnt)$ 。

坑点： 第三维k记录"前面跟了几个同色盒子"，是区间DP的扩展维度。 $O(n^4)$ 。

代码：

```

1  class Solution {
2  public:
3      // 时间复杂度  $O(n^4)$ 
4      int removeBoxes(vector<int>& boxes) {
5          int n = boxes.size();
6          memset(dp, 0, sizeof(dp));
7          return f(boxes, 0, n - 1, 0);
8      }
9
10     // boxes[l...r] 范围上去消除，前面跟着k个连续的和boxes[l]颜色一样的盒子
11     // 这种情况下，返回最大得分
12     int f(vector<int>& boxes, int l, int r, int k) {
13         if (l > r) {
14             return 0;
15         }
16         // l <= r
17         if (dp[l][r][k] > 0) {
18             return dp[l][r][k];
19         }
20         int s = l;

```

```

21     while (s + 1 <= r && boxes[l] == boxes[s + 1]) {
22         s++;
23     }
24     // boxes[l...s]都是一种颜色, boxes[s+1]就不是同一种颜色了
25     // cnt是总前缀数量 : 之前的相同前缀(k个) + l...s这个颜色相同的部分(s-l+1个)
26     int cnt = k + s - l + 1;
27     // 可能性1 : 前缀先消
28     int ans = cnt * cnt + f(boxes, s + 1, r, 0);
29     // 可能性2 : 讨论前缀跟着哪个后, 一起消掉
30     for (int m = s + 2; m <= r; m++) {
31         if (boxes[l] == boxes[m] && boxes[m - 1] != boxes[m]) {
32             // boxes[l] == boxes[m]是必须条件
33             // boxes[m - 1] != boxes[m]是剪枝条件, 避免不必要的调用
34             ans = max(ans, f(boxes, s + 1, m - 1, 0) + f(boxes, m, r, cnt));
35         }
36     }
37     dp[l][r][k] = ans;
38     return ans;
39 }
40 };
41

```

05 合并石头的最低成本 — LeetCode 1000

题面： n堆石头，每次将连续k堆合并为1堆，成本为这k堆的石子总数。求合并成1堆的最低成本，不能则返回-1。

核心思路： 前提： $(n-1)\%(k-1)==0$ 才能合成1堆。 $dp[l][r] = 1..r$ 合并到不能再合并的最小成本。枚举m（步长k-1）： $dp[l][r]=\min(dp[l][m]+dp[m+1][r])$ 。若 $(r-l)\%(k-1)==0$ （最终能成1份），额外加上 $sum[1..r]$ 。
m跳k-1保证左右子问题的份数约束。

坑点： 合并前必须先检查 $(n-1)\%(k-1)==0$ ；枚举m的步长必须是k-1。

代码：

```

1  int mergeStones(vector<int>& stones, int k) {
2      int n = stones.size();
3      if ((n-1)%(k-1) != 0) return -1;
4      presum[0] = 0;
5      for (int i = 0; i < n; i++) presum[i+1] = presum[i] + stones[i];
6      memset(dp, 0, sizeof(dp));
7      for (int l = n-2; l >= 0; l--)
8          for (int r = l+1; r < n; r++) {
9              int ans = INT_MAX;
10             for (int m = l; m < r; m += k-1)
11                 ans = min(ans, dp[l][m] + dp[m+1][r]);
12             if ((r-l)%(k-1) == 0)
13                 ans += presum[r+1] - presum[l];
14             dp[l][r] = ans;
15         }
16     return dp[0][n-1];
17 }

```

06 统计不同回文子序列 — LeetCode 730

题面：给定字符串s，返回**不同的**非空回文子序列个数（对 $1e9+7$ 取模）。

核心思路： $O(n^2)$ 。 $dp[i][j]$ = s[i..j] 上不同回文子序列个数。预处理 $left[i] = i$ 左边最近同字符位置， $right[i] = i$ 右边最近同字符位置。

若 $s[i] \neq s[j]$ ： $dp[i][j] = dp[i][j-1] + dp[i+1][j] - dp[i+1][j-1]$ （容斥）

若 $s[i] = s[j]$ （设为a）：分三种情况——

- 内部无a： $2 * dp[i+1][j-1] + 2$ （a、aa两个新的）
- 内部有1个a： $2 * dp[i+1][j-1] + 1$ （aa一个）
- 内部 ≥ 2 个a： $2 * dp[i+1][j-1] - dp[l+1][r-1]$ （aa重复计算了）

坑点：减法取模要+MOD再%MOD；left/right预处理是关键。

代码：

```
1  int countPalindromicSubsequences(string str) {
2      int n = str.length();
3      // 预处理 left/right
4      memset(last, -1, sizeof(last));
5      for (int i = 0; i < n; i++) { leftIdx[i]=last[str[i]]; last[str[i]]=i; }
6      for (int i = 0; i < 256; i++) last[i] = n;
7      for (int i = n-1; i >= 0; i--) { rightIdx[i]=last[str[i]]; last[str[i]]=i; }
8      for (int i = 0; i < n; i++) dp[i][i] = 1;
9      for (int i = n-2; i >= 0; i--)
10         for (int j = i+1; j < n; j++) {
11             if (str[i] != str[j])
12                 dp[i][j] = dp[i][j-1] + dp[i+1][j] - dp[i+1][j-1] + MOD;
13             else {
14                 int l = rightIdx[i], r = leftIdx[j];
15                 if (l > r) dp[i][j] = dp[i+1][j-1]*2 + 2;
16                 else if (l == r) dp[i][j] = dp[i+1][j-1]*2 + 1;
17                 else dp[i][j] = dp[i+1][j-1]*2 - dp[l+1][r-1] + MOD;
18             }
19             dp[i][j] %= MOD;
20         }
21     return (int)dp[0][n-1];
22 }
```

class078 树型DP（上）

B站视频：[算法讲解078](#)

前置知识：class037（二叉树递归）、class067/068（二维DP）

01 最大BST子树 — LeetCode 333

题面：二叉树，找节点数最多的二叉搜索子树，返回其大小。

核心思路： 树型DP套路——每节点返回Info (maxVal, minVal, isBst, maxBstSize)。整合：左子树+右子树都BST且左max<x.val<右min → 当前子树是BST。空节点返回 (LLONG_MIN, LLONG_MAX, true, 0)。

坑点： 空节点的max/min值为"无限小/无限大"，让所有正常节点都满足BST条件。节点值可能等于INT_MIN/MAX，所以用 long long。

代码：

```
1 struct Info { long long maxVal, minVal; bool isBst; int maxBstSize; };
2 Info f(TreeNode* x) {
3     if (!x) return Info(LLONG_MIN, LLONG_MAX, true, 0);
4     Info l = f(x->left), r = f(x->right);
5     long long mx = max((long long)x->val, max(l.maxVal, r.maxVal));
6     long long mn = min((long long)x->val, min(l.minVal, r.minVal));
7     bool bst = l.isBst && r.isBst && l.maxVal < x->val && x->val < r.minVal;
8     int sz = bst ? l.maxBstSize + r.maxBstSize + 1
9                 : max(l.maxBstSize, r.maxBstSize);
10    return Info(mx, mn, bst, sz);
11 }
```

02 二叉搜索子树的最大键值和 — LeetCode 1373

题面： 同01，但返回最大键值和 (sum) 而非节点数。

核心思路： Info多加一个sum字段。整合时：若当前子树是BST，maxBstSum = max(左max, 右max, 当前sum)。

坑点： 节点值范围在 [-40000, 40000]，INT_MIN/MAX够用，可以不用long long。

代码：

```
1 struct Info { int maxVal, minVal, sum, maxBstSum; bool isBst; };
2 Info f(TreeNode* x) {
3     if (!x) return Info(INT_MIN, INT_MAX, 0, 0, true);
4     Info l = f(x->left), r = f(x->right);
5     int mx = max(x->val, max(l.maxVal, r.maxVal));
6     int mn = min(x->val, min(l.minVal, r.minVal));
7     int sum = l.sum + r.sum + x->val;
8     bool bst = l.isBst && r.isBst && l.maxVal < x->val && x->val < r.minVal;
9     int mbs = max(l.maxBstSum, r.maxBstSum);
10    if (bst) mbs = max(mbs, sum);
11    return Info(mx, mn, sum, mbs, bst);
12 }
```

03 二叉树的直径 — LeetCode 543

题面： 二叉树上任意两节点间最长路径的边数。

核心思路： Info = (diameter, height)。height = max(左高, 右高)+1。diameter = max(左直径, 右直径, 左高+右高)。

坑点： 直径不一定过根节点。

代码：

```
1 struct Info { int diameter, height; };
2 Info f(TreeNode* x) {
3     if (!x) return Info(0, 0);
4     Info l = f(x->left), r = f(x->right);
5     int h = max(l.height, r.height) + 1;
6     int d = max({l.diameter, r.diameter, l.height + r.height});
7     return Info(d, h);
8 }
```

04 在二叉树中分配硬币 — LeetCode 979

题面： n 个节点的树共有 n 枚硬币（分布在节点上）。一次移动将一枚硬币从节点移到相邻节点。求让每个节点恰好 1 枚硬币的最少移动次数。

核心思路： $\text{Info} = (\text{cnt}, \text{sum}, \text{move})$ 。子树的节点数 vs 硬币数的差值 = 需要进出该子树的硬币数。 $\text{moves} = \text{左move} + \text{右move} + \text{abs}(\text{左cnt} - \text{左sum}) + \text{abs}(\text{右cnt} - \text{右sum})$ 。

坑点： 核心洞察——每个子树“缺多少”或“多多少”硬币，都要通过父边运输，贡献 = $|\text{cnt} - \text{sum}|$ 步。

代码：

```
1 struct Info { int cnt, sum, move; };
2 Info f(TreeNode* x) {
3     if (!x) return Info(0, 0, 0);
4     Info l = f(x->left), r = f(x->right);
5     int cnts = l.cnt + r.cnt + 1;
6     int sums = l.sum + r.sum + x->val;
7     int moves = l.move + r.move + abs(l.cnt - l.sum) + abs(r.cnt - r.sum);
8     return Info(cnts, sums, moves);
9 }
```

05 没有上司的舞会 — 洛谷 P1352 (LeetCode 337 打家劫舍 III)

题面： 树形组织结构，每个职员有快乐值。若上司参加，其直接下属就不参加。求最大总快乐值。

核心思路： $\text{no}[u]$ = u 不来时子树最大快乐值， $\text{yes}[u]$ = u 来时最大。递推： $\text{no}[u] = \sum \max(\text{no}[v], \text{yes}[v])$ ， $\text{yes}[u] = \text{happy}[u] + \sum \text{no}[v]$ 。最终答案 = $\max(\text{no}[\text{root}], \text{yes}[\text{root}])$ 。

坑点： 用链式前向星建图；先找根节点（boss 标记）。

代码：

```

1 void f(int u) {
2     no[u] = 0; yes[u] = nums[u];
3     for (int ei = head[u]; ei > 0; ei = nxt[ei]) {
4         int v = to[ei]; f(v);
5         no[u] += max(no[v], yes[v]);
6         yes[u] += no[v];
7     }
8 }

```

06 监控二叉树 — LeetCode 968

题面：节点上装摄像头可监视自身、父、子。求覆盖整棵树的最少摄像头数。

核心思路：递归（假设x上方一定有父亲！）。返回值含义：

- 0：x无覆盖，但下方都覆盖了
- 1：x有覆盖（无摄像头），下方都覆盖
- 2：x有覆盖（有摄像头），下方都覆盖

转移：①左或右=0 → 必须放摄像头，返回2；②左=1且右=1 → 指望父亲覆盖，返回0；③否则返回1。最后若根节点返回0，再+1。

坑点：假设"x上方一定有父亲"让空节点返回1（被覆盖状态）；根节点无父亲，需最后补判。

代码：

```

1 int f(TreeNode* x) {
2     if (!x) return 1;
3     int l = f(x->left), r = f(x->right);
4     if (l == 0 || r == 0) { ans++; return 2; }
5     if (l == 1 && r == 1) return 0;
6     return 1;
7 }
8 int minCameraCover(TreeNode* root) {
9     ans = 0;
10    if (f(root) == 0) ans++;
11    return ans;
12 }

```

07 路径总和 III — LeetCode 437

题面：二叉树，求从任意节点向下（只能父到子）的路径中，和等于targetSum的路径数。

核心思路："前缀和+HashMap"在树上的应用。DFS时维护"根到当前节点的前缀和"以及前缀和出现次数的哈希表。以当前节点作为路径结尾，`ans += presum[sum - target]`。回溯时恢复哈希表（`presum[sum]--`）。

坑点：初始化 `presum[0] = 1`（路径从根开始的情况）；回溯时撤销计数。

代码：

```

1 void f(TreeNode* x, int target, long long sum) {
2     if (!x) return;
3     sum += x->val;
4     ans += presum[sum - target];
5     presum[sum]++;
6     f(x->left, target, sum);
7     f(x->right, target, sum);
8     presum[sum]--; // 回溯
9 }

```

class079 树型DP（下） — 树型DP和树上背包

B站视频: [算法讲解079](#)

前置知识: class078 (树型DP上)、class073 (01背包)

01 到达首都的最少油耗 — LeetCode 2477

题面: n 个城市形成树, 0是首都, 每个城市有1个代表。每辆车seats个座位, 相邻城市间行驶耗1升油。求所有人抵达首都的最少油耗。

核心思路: DFS后序遍历。size[u] = 子树人数, cost[u] = 子树内部总油耗。每个子节点v到u的边需要 $\text{ceil}(\text{size}[v]/\text{seats})$ 辆车, 即 $(\text{size}[v] + \text{seats} - 1) / \text{seats}$ 趟。 $\text{cost}[u] = \sum(\text{cost}[v] + \text{趟数})$ 。

坑点: 无向图用parent参数防回头; 整除向上取整写法 $(a+b-1)/b$ 。

代码:

```

1 void dfs(int seats, int u, int p) {
2     sz[u] = 1;
3     for (int v : graph[u]) {
4         if (v != p) {
5             dfs(seats, v, u);
6             sz[u] += sz[v];
7             cost[u] += cost[v] + (sz[v] + seats - 1) / seats;
8         }
9     }
10 }

```

02 相邻字符不同的最长路径 — LeetCode 2246

题面: 树 (0为根), 每个节点有一个字符。找最长路径满足: 路径上任意相邻节点字符不同。返回节点数。

核心思路: Info = (maxPathFromHead, maxPath)。对每个孩子v, 若 $s[u] \neq s[v]$, 用其maxPathFromHead更新max1/max2 (最长链/次长链)。最终: $\text{maxPathFromHead} = \text{max1} + 1$, $\text{maxPath} = \max(\text{孩子maxPath}, \text{max1} + \text{max2} + 1)$ 。

坑点: 只合并字符不同的孩子; 叶子返回 (1, 1)。

代码:


```

1 struct Info { int maxPathFromHead, maxPath; };
2 Info f(string& s, int u) {
3     if (graph[u].empty()) return Info(1, 1);
4     int max1 = 0, max2 = 0, maxPath = 1;
5     for (int v : graph[u]) {
6         Info next = f(s, v);
7         maxPath = max(maxPath, next.maxPath);
8         if (s[u] != s[v]) {
9             if (next.maxPathFromHead > max1) { max2 = max1; max1 =
next.maxPathFromHead; }
10            else if (next.maxPathFromHead > max2) max2 = next.maxPathFromHead;
11        }
12    }
13    return Info(max1 + 1, max(maxPath, max1 + max2 + 1));
14 }

```

03 移除子树后的二叉树高度 — LeetCode 2458

题面： 二叉树， m 个独立查询：每次移除以某值为根的子树（不修改原树），求树剩余部分的高度。

核心思路： DFN序。DFS时给每个节点分配序号 `dfn`，记录深度 `deep[dfn]` 和子树大小 `sz[dfn]`。子树在DFN序中是一段连续区间。预计算 `maxl[i]`=DFN[0..i]的最大深度，`maxr[i]`=DFN[i..end]的最大深度。对于查询 v ：答案 $= \max(\maxl[dfn[v]-1], \maxr[dfn[v]+sz[dfn[v]]])$ 。

坑点： 查询不修改原树， $O(1)$ 回答；深度是边数。

代码：

```

1 void dfs(TreeNode* x, int k) {
2     int i = ++dfnCnt; dfn[x->val] = i; deep[i] = k; sz[i] = 1;
3     if (x->left) { dfs(x->left, k+1); sz[i] += sz[dfn[x->left->val]]; }
4     if (x->right) { dfs(x->right, k+1); sz[i] += sz[dfn[x->right->val]]; }
5 }
6 // 查询
7 int leftMax = maxl[dfn[q] - 1];
8 int rightMax = maxr[dfn[q] + sz[dfn[q]]];
9 ans = max(leftMax, rightMax);

```

04 从树中删除边的最小分数 — LeetCode 2322

题面： 树，每个节点有权值。删两条边形成三个连通分量，分数=最大异或值-最小异或值。求最小分数。

核心思路： DFN序+XOR。DFS求子树XOR值 `xorVal[dfn]`。枚举所有边对($O(m^2)$)，取较大DFN端点。两棵子树的关系：① pos 在 pre 子树内 $\rightarrow$ `sum1=xor[pos]`，`sum2=xor[pre]^xor[pos]`，`sum3=全部XOR^xor[pre]`；②独立 $\rightarrow$ `sum1=xor[pos]`，`sum2=xor[pre]`，`sum3=全部XOR^sum1^sum2`。

坑点： 判断两个子树是否嵌套：`pos < pre + sz[pre]`。

代码：

```

1 for (int i = 0; i < m; i++) {

```

```

2   a = max(dfn[edges[i][0]], dfn[edges[i][1]]);
3   for (int j = i+1; j < m; j++) {
4       b = max(dfn[edges[j][0]], dfn[edges[j][1]]);
5       pre = min(a,b); pos = max(a,b);
6       sum1 = xorVal[pos];
7       if (pos < pre + sz[pre]) {
8           sum2 = xorVal[pre] ^ xorVal[pos];
9           sum3 = xorVal[1] ^ xorVal[pre];
10      } else {
11          sum2 = xorVal[pre];
12          sum3 = xorVal[1] ^ sum1 ^ sum2;
13      }
14      ans = min(ans, max({sum1,sum2,sum3}) - min({sum1,sum2,sum3}));
15  }
16  }

```

05 选课（树上01背包） — 洛谷 P2014

题面： n 门课，每门课可能有先修课。选 m 门课获得最大学分。先修关系构成树（0号虚拟节点为根）。

普通解法 $O(n \cdot \text{avgChildren} \cdot m^2)$

三维DP。 $f(i, j, k)$ = 节点 i 和它的前棵子树，选 k 个节点（连通）的最大累加和。跳过第 j 棵子树 $\rightarrow f(i, j-1, k)$ ；从第 j 棵子树选 s 个 $\rightarrow f(i, j-1, k-s) + f(v, \text{allChildren}, s)$ 。

最优解 $O(n \cdot m)$ — DFN序+01背包

核心洞察： DFN序上，若跳过节点 i ，则必须跳过它的整棵子树（不能选后代不选祖先）。 $dp[i][j]$ = DFN从 i 到尾，选 j 个节点形成"有效结构"（挂在假想根下连通无断）的最大和。

转移：①跳过 $i \rightarrow dp[i+sz[i]][j]$ （跳过整个子树）；②选 $i \rightarrow val[i] + dp[i+1][j-1]$ 。

最终答案： $nums[0] + dp[2][m]$ （0号虚拟节点在DFN=1，必然选）。

坑点： 这是树上01背包的最优解法，理解"有效结构"定义是关键。

代码（最优解）：

```

1  int dfs(int u) {
2      int i = ++dfnCnt; val[i] = nums[u]; sz[i] = 1;
3      for (int ei = head[u]; ei > 0; ei = nxt[ei])
4          sz[i] += dfs(to[ei]);
5      return sz[i];
6  }
7  int compute() {
8      dfs(0); // n+1个节点, DFN 1..n+1
9      for (int i = n+1; i >= 2; i--)
10         for (int j = 1; j <= m; j++)
11             dp[i][j] = max(dp[i + sz[i]][j],           // 不要节点i（跳过整个子树）
12                             val[i] + dp[i + 1][j - 1]); // 要节点i
13      return nums[0] + dp[2][m];
14  }

```

总结：树型DP笔记

题目	核心技巧	类型
到达首都油耗	后序DFS + size/cost累积	树型DP
相邻不同最长路径	Info合并 + 最长/次长链	树型DP
移除子树高度	DFN序 + 前后缀max	DFN技巧
删边最小分数	DFN序 + XOR + 嵌套判断	DFN技巧
选课（最优解）	DFN序 + 01背包转换 $O(nm)$	树上背包

class080 状压DP（上）

B站视频：[算法讲解080](#)
前置：位运算（003/030-033）、根据数据量猜解（043）、双向广搜（063）、递归→DP（067）

核心知识：状压DP

是什么

用整数的**位信息**表示某些样本"是否可用/是否用过"。k个样本 → 状态数 2^k ，通常 $k \leq 20$ 才可行（ $2^{20} \approx 10^6$ ）。每个状态0/1表示对应样本的可用性。

什么时候用 / 识别特征

- 问题的每一步从集合中选一个元素，选过的不能再选
- $N \leq 20$ 且有"是否使用过"的状态需要记住
- 暴力递归会重复计算相同状态（如：先选 A 再选 B 和先选 B 再选 A，后续状态相同）
- 数据量判断：** $2^N \times \text{枚举代价}$ 不超过 $10^7 \sim 10^8$ 就可以用状压DP

核心套路

递归函数 → 挂记忆化缓存 `dp[status]` → 必要时改严格位置依赖DP → 空间压缩。只对**关键可变参数**做缓存——被决定的参数（如累计和）不用管。

01 我能赢吗 — LeetCode 464

题面： 1~n的数字池，两人轮流取数（不放回），累加和 $\geq m$ 者获胜。问先手能否稳赢。

核心思路： `status` 的第 i 位=1 表示数字 i 还能选。遍历所有可选的数字，若存在一个数字使得对手后续必输（`!f(新status, rest-i)`），则自己赢。记忆化缓存 `dp[status]`。

关键洞察——为什么缓存只对status做： `rest` 被 `status` 唯一决定——status确定了哪些数被挑走了，累加和也就确定了。反过来不成立（不同挑法可能剩余相同rest）。所以只缓存status。

状态数量： $2^{(n+1)}$ （多留一个0位弃而不用），每次枚举n个数字 → $O(2^n \times n)$ 。

坑点： $m==0$ 直接true；总和 $<m$ 直接false。

代码：

```
1 bool f(int n, int status, int rest) {
2     if (rest <= 0) return false;
3     if (dp[status] != 0) return dp[status] == 1;
4     for (int i = 1; i <= n; i++)
5         if ((status & (1 << i)) && !f(n, status ^ (1 << i), rest - i))
6             return dp[status] = 1, true;
7     return dp[status] = -1, false;
8 }
```

02 火柴拼正方形 — LeetCode 473

题面： 用所有火柴拼正方形（4条等边），不能折断，必须全用。

核心思路： 策略：依次冲第一条边→第二条→第三条→第四条，当前边不凑满不开下一条。

- `status`：哪些火柴还能用（1=可用）
- `cur`：当前边已拼长度
- `rest`：还剩几条边

每根火柴：能用 + 不超边长 → ①恰好凑满： `cur=0, rest-1`；②没满： `cur+len, rest` 不变。

关键洞察——为什么dp只对status做： `cur` 和 `rest` 被 `status` 唯一决定！因为策略是从不超边长的，`status` 定了→已用火柴总长定了→当前边进度和完成边数也就定了。

代码：

```
1 bool f(vector<int>& nums, int limit, int status, int cur, int rest) {
2     if (rest == 0) return status == 0;
3     if (dp[status] != 0) return dp[status] == 1;
4     for (int i = 0; i < n; i++)
5         if ((status & (1 << i)) && cur + nums[i] <= limit) {
6             if (cur + nums[i] == limit)
7                 { if (f(nums, limit, status^(1<<i), 0, rest-1)) return
8 dp[status]=1,true; }
9             else
10                { if (f(nums, limit, status^(1<<i), cur+nums[i], rest)) return
11 dp[status]=1,true; }
12        }
13    return dp[status] = -1, false;
14 }
```

03 划分为k个相等的子集 — LeetCode 698

题面： 同02，但边数改为k（不限于4）。

核心思路： 状压DP版——递归函数与02完全相同，只改主函数参数（k条边、边长=sum/k）。

回溯+剪枝版（纯暴力）： 建k个桶，每个数字尝试放每个桶（不超过target），大数先放（排序后从后往前）。剪枝策略：①当前桶超target的跳过；②如果相邻桶的累加和相同且当前数字在该桶已失败，跳过相同累加和的桶（因为状态等价）。复杂度 $O(K^N)$ ，靠剪枝硬过——数据不苛刻时可能比状压DP还快，但不稳定。

代码：同02，略。

04 TSP售货员问题 — 洛谷 P1171

题面： n个村庄，有向带权图。从1号出发，每个村走一次仅一次，最后回1号。求最短路程。

核心思路： $f(s, i)$ = 已走村庄状态s (1=走过)，当前在i村，走完剩余并回0的最短距离。

- base case: $s == (1 \ll n) - 1$ (全走过) → 直接回0号村，返回 $graph[i][0]$
- 转移：枚举每个未走过的村庄j, $f(s | 1 \ll j, j) + graph[i][j]$ ，取min

状态空间： $2^n \times n$, $n \leq 19$ 可过C++ (Java可能卡空间)。空间优化版将起始村"抠掉", n减1, 状态数减半。

代码：

```
1 int f(int s, int i) {
2     if (s == (1 << n) - 1) return graph[i][0];
3     if (dp[s][i] != -1) return dp[s][i];
4     int ans = INT_MAX;
5     for (int j = 0; j < n; j++)
6         if ((s & (1 << j)) == 0)
7             ans = min(ans, graph[i][j] + f(s | (1 << j), j));
8     return dp[s][i] = ans;
9 }
```

class081 状压DP（下）

B站视频：[算法讲解081](#)

前置：class080（状压DP上）

核心知识

本节课展示状压DP与多种技巧的组合：反转思路（谁做状态）、枚举子集、质因子打表、Brian Kernighan 逐位提取。

关键枚举技巧：遍历集合的所有子集

```
1 for (int sub = status; sub > 0; sub = (sub - 1) & status) {
2     // sub 是 status 的一个非空子集
3 }
```

01 每个人戴不同帽子的方案数 — LeetCode 1434

题面： n人(≤ 10)，40种帽子。每人有喜好列表，一人一顶不同帽，求方案数。

核心思路——反转思考：若对帽子状压 2^{40} 太大。反转——**对人状压：** $n \leq 10$ ，状态仅 2^{10} 。预处理 `hats[i]` = i号帽子能满足哪些人（位图）。

递归：`f(i, s)` = 从i号帽子出发，已满足的人状态为s，有多少种方案。

- 可能性1：i号帽子不分配给任何人 $\rightarrow f(i+1, s)$
- 可能性2：i号帽子分配给某个它能满足且还没帽子的人 $\rightarrow$ 累加 `f(i+1, s | 1 << p)`

优化 (f2)：用 Brian Kernighan 算法逐位提取帽子能满足的人，跳过 for 循环遍历所有人。`rightOne = cur & -cur` 提取最右侧1，处理后 `cur ^= rightOne` 去掉该位。

坑点：帽子编号1~40，递归从1到m；`hats[i]` 的人位是从0开始。

代码：

```
1 int f(int m, int n, int i, int s) {
2     if (s == (1 << n) - 1) return 1;
3     if (i == m + 1) return 0;
4     if (dp[i][s] != -1) return dp[i][s];
5     int ans = f(m, n, i + 1, s); // 不分配这顶帽子
6     int cur = hats[i];
7     while (cur) {
8         int rightOne = cur & -cur;
9         if ((s & rightOne) == 0)
10             ans = (ans + f(m, n, i + 1, s | rightOne)) % MOD;
11         cur ^= rightOne;
12     }
13     return dp[i][s] = ans;
14 }
```

02 最优账单平衡 — LeetCode 465

题面：给定转账交易，求还清所有债务的最少交易笔数。

核心思路——分两步：

第一步：算出每个人的净收支，去掉为0的。问题变成——把这些人分成尽量多的"不可再拆分组"，每组内部净收支之和=0。最少交易数 = n - 最大组数。

第二步：状压DP求"集合最多能分成多少和为0的不可拆分组"。

- 若集合累加和=0：随便移除一个元素，递归求剩余集合最多能分的组数，然后+1。**不管移除哪个元素结论都一样**，所以试第一个就break。
- 若集合累加和≠0：枚举移除每个元素，取剩余部分能分的最大组数（不加1）。

为什么只用对set做缓存：sum 被 set 决定——set 确定后累加和就是确定的。

坑点：单元素集合的sum≠0（因为数组里没0），直接返回0。判断"集合是否只有一个元素"用 `set & (set-1) == 0`。

代码：

```
1 int f(vector<int>& debt, int set, int sum, int n) {
2     if (dp[set] != -1) return dp[set];
```

```

3     int ans = 0;
4     if ((set & (set - 1)) != 0) { // 不只一个元素
5         if (sum == 0) {
6             for (int i = 0; i < n; i++)
7                 if (set & (1 << i))
8                     return dp[set] = f(debt, set ^ (1 << i), sum - debt[i], n) + 1;
9         } else {
10            for (int i = 0; i < n; i++)
11                if (set & (1 << i))
12                    ans = max(ans, f(debt, set ^ (1 << i), sum - debt[i], n));
13        }
14    }
15    return dp[set] = ans;
16 }

```

03 好子集的数目 — LeetCode 1994

题面：好子集=所有元素乘积的每种质因子只出现一次。求不同好子集个数。

核心思路：1~30以内只有10个质数{2,3,5,7,11,13,17,19,23,29}。**打表** `own[i]`：数字i的质因子位图。含平方因子（如4,8,9,12）的数字标记为0（无效）。

数字1特权：1不含任何质因子。有`cnt1`个1，它们可以任意组合附加到任何好子集上，贡献 2^{cnt1} 倍。

递归：`f(i, s)` = 从数字i往下，要凑出质因子状态恰好为s，有多少种子集。从30递减到1：

- 不要i $\rightarrow$ `f(i-1, s)`
- 要i (条件: `own[i]≠0` 且 `(s & own[i]) == own[i]`) $\rightarrow$ `cnt[i] × f(i-1, s^own[i])`

空间压缩：依赖上一行和自己左上方，从右往左一维数组更新。

坑点：最终答案累加所有 `s ≠ 0` 的 `dp[s]`；取模时减法先+MOD。

代码（空间压缩版）：

```

1  dp[0] = 1;
2  for (int j = 0; j < cnt[1]; j++) dp[0] = (dp[0] << 1) % MOD;
3  for (int i = 2; i <= 30; i++) {
4      int cur = own[i], times = cnt[i];
5      if (cur && times)
6          for (int s = LIMIT - 1; s >= 0; s--)
7              if ((s & cur) == cur)
8                  dp[s] = (1LL * dp[s ^ cur] * times + dp[s]) % MOD;
9  }
10 int ans = 0;
11 for (int s = 1; s < LIMIT; s++) ans = (ans + dp[s]) % MOD;

```

04 分配重复整数 — LeetCode 1655

题面：数组中若干种数（每种有出现次数），m个顾客各要 `quantity[i]` 个相同数字。问能否满足。

核心思路：统计每种数字的个数 `cnt[]`。预处理 `sum[mask]` = 顾客子集需要多少个数字。

状压DP: $f(\text{status}, \text{index})$ = 用第 index 种及之后的数字, 能否满足剩余订单 status 。

核心枚举: 遍历 status 的所有子集, 尝试用当前数字的个数一次满足某个子集:

```
1 for (int sub = status; sub > 0; sub = (sub - 1) & status)
2     if (sum[sub] <= cnt[index] && f(status ^ sub, index + 1))
3         return true;
```

若所有子集都不行, 再尝试"跳过当前数字"。

坑点: 枚举子集 $\text{sub} = (\text{sub}-1) \& \text{status}$ 是状压DP的必背模板。时间复杂度 $O(n \times 3^m)$ (枚举子集的所有子集)。

代码:

```
1 bool f(vector<int>& cnt, int status, int index) {
2     if (status == 0) return true;
3     if (index == cnt.size()) return false;
4     if (dp[status][index] != 0) return dp[status][index] == 1;
5     int k = cnt[index];
6     for (int sub = status; sub > 0; sub = (sub - 1) & status)
7         if (sum[sub] <= k && f(cnt, status ^ sub, index + 1))
8             return dp[status][index] = 1, true;
9     if (f(cnt, status, index + 1)) return dp[status][index] = 1, true;
10    return dp[status][index] = -1, false;
11 }
```

class082 动态规划中用观察优化枚举的技巧 (上)

B站视频: [算法讲解082](#)

前置: class066-081 动态规划系列

核心知识

动态规划复杂度 = 状态数 $\times$ 每个状态的枚举代价。记忆化搜索在一个格子需要走for循环枚举若干子过程时, 可能不是最优解。改出**严格位置依赖的动态规划**建立空间感后, 通过**观察转移方程**可以发现相邻状态的枚举有重叠, 从而用**窗口累加和**或**best变量**替代枚举, 将枚举代价降为 $O(1)$ 。

本节课重点: 股票问题系列 (1-6) + DI序列 (题目7)。**题目4 (股票k次交易)**和**题目7**最能体现观察优化枚举的技巧。

01 买卖股票的最佳时机 — LeetCode 121

题面: 只能买卖一次, 求最大利润。

核心思路: 枚举每个点作为卖出时机。遍历时维护 $0..i$ 的最小值 min , 当天利润 = $\text{prices}[i] - \text{min}$ 。

代码:


```

1 int ans = 0;
2 for (int i = 1, minVal = prices[0]; i < n; i++) {
3     minVal = min(minVal, prices[i]);
4     ans = max(ans, prices[i] - minVal);
5 }

```

02 买卖股票的最佳时机 II — LeetCode 122

题面：无限次交易，同一时刻最多持有一股。

核心思路：抓住所有上坡。 `ans += max(prices[i]-prices[i-1], 0)`。上坡分批累加等价于一次完整的低买高卖。

代码：

```

1 int ans = 0;
2 for (int i = 1; i < n; i++)
3     ans += max(prices[i] - prices[i - 1], 0);

```

03 买卖股票的最佳时机 III — LeetCode 123

题面：最多完成两笔交易。

核心思路： `dp2[i]` = 0..i范围上发生两次交易，且第二次在i时刻卖出的最大利润。

方法1 (超时, $O(n^2)$) : `dp2[i] = max(dp1[j] + prices[i] - prices[j])`, $j \in [0, i]$ 枚举第二次买入时机。

方法2 (观察优化, $O(n)$) : 把式子改写: `dp2[i] = prices[i] + max(dp1[j] - prices[j])`。引入 `best[i] = max(dp1[k] - prices[k]) for  $k \leq i$` , 则 `dp2[i] = best[i] + prices[i]`。 `best[i]`从前一步 `best[i-1]`和当前新指标 `dp1[i]-prices[i]` 取max即可。

方法3→4 (合并+空间压缩) : 所有更新塞一个循环，用滚动变量替代数组。

代码 (最终版) :

```

1 int dp1 = 0, best = -prices[0], ans = 0;
2 for (int i = 1, minVal = prices[0]; i < n; i++) {
3     minVal = min(minVal, prices[i]);
4     dp1 = max(dp1, prices[i] - minVal);
5     best = max(best, dp1 - prices[i]);
6     ans = max(ans, best + prices[i]);
7 }

```

04 买卖股票的最佳时机 IV — LeetCode 188 ★核心模板

题面：最多k笔交易。

核心思路：剪枝: `k >= n/2` → 等价无限次 (题目2)。

DP定义: `dp[i][j]` = 0..j范围上进行i次交易的最大利润。转移:

```

1 dp[i][j] = max( dp[i][j-1], // j不参与
2               max_{p<j} dp[i-1][p] - prices[p] + prices[j] ) // j为卖出时机

```

用 `best` 变量维护 `max(dp[i-1][p] - prices[p])`：初始 `best = dp[i-1][0] - prices[0]`，每步 `best = max(best, dp[i-1][j] - prices[j])`，则 `dp[i][j] = max(dp[i][j-1], best + prices[j])`。

空间压缩（最终版）：

```

1 if (k >= n / 2) return free(prices);
2 memset(dp, 0, sizeof(dp));
3 for (int i = 1, best, tmp; i <= k; i++) {
4     best = dp[0] - prices[0];
5     for (int j = 1; j < n; j++) {
6         tmp = dp[j];
7         dp[j] = max(dp[j - 1], best + prices[j]);
8         best = max(best, tmp - prices[j]);
9     }
10 }
11 return dp[n - 1];

```

坑点： 枚举优化和股票3完全一样——都是把 `dp[i-1][p] - prices[p]` 的最大值用滚动变量维护，省掉内层for。

05 买卖股票含手续费 — LeetCode 714

题面： 无限次交易，每笔卖出扣手续费`fee`。

核心思路： 状态机：

- `prepare`：获得收益的同时已扣掉一次购买+手续费的最优情况
- `done`：无限次交易的最大收益

转移：`done = max(done, prepare + prices[i])`（今天卖出），`prepare = max(prepare, done - prices[i] - fee)`（今天买入或保持）。

代码：

```

1 int prepare = -prices[0] - fee, done = 0;
2 for (int i = 1; i < n; i++) {
3     done = max(done, prepare + prices[i]);
4     prepare = max(prepare, done - prices[i] - fee);
5 }

```

06 买卖股票含冷冻期 — LeetCode 309

题面： 卖出后冷冻1天不能买入。

核心思路： `prepare[i]` = 0..i上无限次交易+必须扣一次购买的最优。`done[i]` = 0..i上无限次交易的最大收益。因为有冷冻期，`prepare[i] = max(prepare[i-1], done[i-2] - prices[i])`（必须等一天）。

空间压缩： 只需 `prepare`, `done2` (`done[i-2]`), `done1` (`done[i-1]`) 三个变量滚动。

07 DI序列的有效排列 — LeetCode 903 ★观察优化枚举

题面：字符串s由D/I组成，用0~n的数字排列满足D=下降、I=上升关系，求方案数。

核心思路（巧妙的状态设计）： $f(i, less)$ = 来到位置i，前一个数字已确定但不知道是谁，只知道还没用过的数字中比前一个小的有less个。不需要知道具体哪些数字用过！

枚举展开：

- 开头或 $s[i-1] == 'D'$ ：下一个选比前一个小的 → 枚举 $nextLess \in [0, less-1]$
- $s[i-1] == 'I'$ ：下一个选比前一个大的 → 枚举 $nextLess \in [less, less+(n-i-less)-1]$

优化枚举（观察法）： $dp[i][less]$ 对nextLess的求和有重叠——

- D时: $dp[i][less] = dp[i][less-1] + dp[i+1][less-1]$ （前缀和）
- I时: $dp[i][less] = dp[i][less+1] + dp[i+1][less]$ （后缀和）

从 $O(n^3)$ 降至 $O(n^2)$ 。

代码：

```
1  for (int less = 0; less <= n; less++) dp[n][less] = 1;
2  for (int i = n - 1; i >= 0; i--) {
3      if (i == 0 || s[i - 1] == 'D') {
4          dp[i][1] = dp[i + 1][0];
5          for (int less = 2; less <= n; less++)
6              dp[i][less] = (dp[i][less - 1] + dp[i + 1][less - 1]) % MOD;
7      } else {
8          dp[i][n - i - 1] = dp[i + 1][n - i - 1];
9          for (int less = n - i - 2; less >= 0; less--)
10             dp[i][less] = (dp[i][less + 1] + dp[i + 1][less]) % MOD;
11     }
12 }
```

class083 动态规划中用观察优化枚举的技巧（下）

B站视频: [算法讲解083](#)

前置: class066-082 动态规划系列

核心知识

上节课讲"观察并优化转移方程"，本节课继续。同时引入"观察并设计高效的查询结构"来优化枚举——题目1用二分查找，题目3用有序数组+二分，题目4用minSums高效结构。

本节课还讲了一个重要的窗口技巧（题目4）：**窗口不回退**——当只关心"最长/最大"答案时，换开头后窗口不需要从头扩，只需要看能不能吸收下一块形成更长的答案。

01 规划兼职工作 — LeetCode 1235

题面： n份工作（开始时间、结束时间、报酬），时间不重叠，求最大报酬。

核心思路：按结束时间排序。 $dp[i] = 0..i$ 份工作的最大报酬。工作 i ：①不做 $\rightarrow dp[i-1]$ ；②做 $\rightarrow profit[i] + dp[find(i-1, start[i])]$ 。find：二分找"结束时间 $\leq start[i]$ 的最右工作"。 $O(n \log n)$ 。

观察点：dp有单调性——范围越大收益只增不减，所以选"尽量靠右的结束时间 $\leq start$ 的工作"最优。二分替代线性枚举。

代码：

```
1 sort(jobs, jobs + n, [](auto& a, auto& b) { return a[1] < b[1]; });
2 dp[0] = jobs[0][2];
3 for (int i = 1; i < n; i++) {
4     int start = jobs[i][0];
5     dp[i] = jobs[i][2];
6     if (jobs[0][1] <= start)
7         dp[i] += dp[find(i - 1, start)];
8     dp[i] = max(dp[i], dp[i - 1]);
9 }
```

02 K个逆序对数组 — LeetCode 629

题面：1~n的排列，求恰好k个逆序对的排列数。

核心思路： $dp[i][j]$ = 用1..i形成恰好j个逆序对的排列数。把i插入到1..(i-1)排列的第几位就增加多少逆序对。

方法1 ($O(n^2k)$ 超时)：内层for枚举p。

方法2 (窗口累加和 $O(nk)$)：画格子观察发现 $dp[i][j]$ 是对上一行某段区间的累加——

- $i > j$ ：窗口只扩不缩， $window += dp[i-1][j]$
- $i \leq j$ ：窗口右扩左缩， $window = window + dp[i-1][j] - dp[i-1][j-i]$

观察点：每个格子的依赖画图后是滑动窗口，用一个window变量维护累加和即可。

代码：

```
1 dp[0][0] = 1;
2 for (int i = 1, window; i <= n; i++) {
3     dp[i][0] = 1; window = 1;
4     for (int j = 1; j <= k; j++) {
5         if (i > j) window = (window + dp[i-1][j]) % MOD;
6         else window = ((window + dp[i-1][j]) % MOD - dp[i-1][j-i] + MOD) % MOD;
7         dp[i][j] = window;
8     }
9 }
```

03 自由之路 — LeetCode 514

题面：环形轮盘ring，每次操作：①按按钮输入当前字符（代价1）；②顺时针/逆时针旋转一格（代价1）。给定key，求输入key的最小代价。

核心思路： $f(i, j)$ = 指针在ring[i]，要输入key[j..]的最小代价。

关键贪心——枚举优化：若 `ring[i]==key[j]` 直接按，不需要移动。否则只需考虑顺时针最近的`key[j]`和逆时针最近的`key[j]`两个位置，中间的其他同名字符一定不会更优！

高效查询结构：预处理 `where[c][k]` = 字符c在ring中的所有位置（有序数组）。顺时针最近→二分找 `>i` 的最左；逆时针最近→二分找 `<i` 的最右。环形距离公式：`distance1 = jump>i ? jump-i : n-i+jump`。

代码：

```
1  if (ring[i] == key[j]) {
2      ans = 1 + f(i, j + 1);
3  } else {
4      int jump1 = clockFind(i, key[j]); // 顺时针最近
5      int d1 = jump1 > i ? jump1 - i : n - i + jump1;
6      int jump2 = counterClockFind(i, key[j]); // 逆时针最近
7      int d2 = i > jump2 ? i - jump2 : i + n - jump2;
8      ans = min(d1 + f(jump1, j), d2 + f(jump2, j));
9  }
```

04 累加和不大于k的最长子数组 — 牛客 ★窗口不回退

题面：数组有正负零，求累加和 $\leq k$ 的最长子数组长度。要求 $O(n)$ 。

核心思路：关键在于"只关心更长的答案"——窗口不回退！

预处理两个结构 ($O(n)$)：

- `minSums[i]`：以i开头往右扩，能得到的最小子数组累加和
- `minSumEnds[i]`：取得该最小值时的结束位置

生成方式（从右往左）：若 `minSums[i+1] < 0`，吞下它更小 → `minSums[i] = nums[i] + minSums[i+1]`，右边界继承；否则自己待着。

主流程（窗口不回退）：从i=0开始，每次尝试把"下一块" (`minSums[end]`) 吞进窗口：若能吞 (`sum+minSums[end] ≤ k`) → 吞入，end跳到下一块末尾+1。吞完后结算当前窗口长度 `end-i`。然后换开头 `i+1`：窗口吐掉`nums[i]`，end不退回！因为"以i+1开头更短的答案我们根本不关心"。

坑点：如果窗口根本没扩出来 (`end==i`)，说明i位置连自己一块都进不来，end跳到i+1跟上i。

代码：

```
1  // 预处理
2  minSums[n-1] = nums[n-1]; minSumEnds[n-1] = n-1;
3  for (int i = n-2; i >= 0; i--) {
4      if (minSums[i+1] < 0)
5          minSums[i] = nums[i] + minSums[i+1],
6          minSumEnds[i] = minSumEnds[i+1];
7      else
8          minSums[i] = nums[i], minSumEnds[i] = i;
9  }
10 // 主流程——窗口不回退
11 int ans = 0;
12 for (int i = 0, sum = 0, end = 0; i < n; i++) {
13     while (end < n && sum + minSums[end] <= k)
```

```

14         sum += minSums[end], end = minSumEnds[end] + 1;
15     ans = max(ans, end - i);
16     if (end > i) sum -= nums[i];
17     else end = i + 1;
18 }

```

总结：优化枚举的两大类

类型	技巧	题目
观察转移方程	best变量替代枚举	股票3、股票4
观察转移方程	窗口累加和	K逆序对、DI序列
设计查询结构	二分查找	工作安排、自由之路
设计查询结构	预处理+窗口不回退	最长子数组

class084 数位DP（上）笔记

视频链接：<https://www.bilibili.com/video/BV1cC4y1Q7r3/>

前置知识：位运算基础（class030-033）、动态规划基础（class066起）

题目1：统计各位数字都不同的数字个数

题面

给定整数 n ，代表十进制数字最多有 n 位。若某个数字的每一位都不同，称为有效数字。返回有效数字的个数（不统计负数范围）。

- 测试链接：<https://leetcode.cn/problems/count-numbers-with-unique-digits/>
- $n=0 \rightarrow$ 返回 1（只有数字 0）

核心思路

直接排列组合，不需要 DP：

- 1 位数：10 种（0~9）
- 2 位数：第一位 1~9（9 种），第二位 0~9 且 ≠ 第一位（9 种） $\rightarrow 9 \times 9$
- 3 位数： $9 \times 9 \times 8$
- k 位数： $9 \times 9 \times 8 \times \dots \times (11-k)$
- 把所有位数的结果累加即可

坑点

- $n=0$ 时返回 1，不是 0
- 第一位不能选 0，但后续位可以选 0

代码

```
1 int countNumbersWithUniqueDigits(int n) {
2     if (n == 0) return 1;
3     int ans = 10;
4     for (int s = 9, i = 9, k = 2; k <= n; i--, k++) {
5         s *= i;
6         ans += s;
7     }
8     return ans;
9 }
```

题目2：最大为N的数字组合

题面

给定有序、无重复、不含 '0' 的字符数组 `digits`（如 `["1","3","5","7"]`），可以任意次使用这些字符拼写数字。返回 $\leq$ 给定整数 n 的正整数个数。

- 测试链接: <https://leetcode.cn/problems/numbers-at-most-n-given-digit-set/>

核心思路

方法一（递归）：从高位到低位线性展开，两个关键状态：

- `free`: 之前是否已经确定比 n 小（1=自由选，0=受限制）
- `fix`: 之前是否选过数字（1=选过，0=一直舍弃）

递归分支：

1. 若 `fix=0`，当前位仍可舍弃（位数比 n 少的全合法）
2. 若 `free=0`，遍历 `digits` 中 $\leq$ 当前位数字的： $<cur$ 则后续 `free`， $=cur$ 继续受限， $>cur$ break
3. 若 `free=1`，`digits` 中全可选，直接 $\times$ 后续可能性

方法二（打表加速）：预处理 `cnt[i]` = 前缀已比 n 小、剩 i 位时有多少种排列（即 m^i ）。位数不够的提前累加到 `ans`。只有数字 $=cur$ 时才递归。

- 复杂度：暴力 $O(m^n)$ （无缓存）， $n \leq 9$ 直接过

坑点

- `offset` 是辅助变量（跟随 `len`，方便提取 n 每位数字），非关键参数
- `digits` 不含 '0'，所以空前缀（`fix=0`）走到 `len=0` 不算有效数字
- 方法二的 `cnt[0]=1` 表示前缀已经确定的一种情况

代码（方法二核心）

```
1  int f2(int num, int offset, int len) {
2      if (len == 0) return 1; // num 自己
3      int cur = (num / offset) % 10;
4      int ans = 0;
5      for (int idx = 0; idx < m; idx++) {
6          int i = dig[idx];
7          if (i < cur) ans += cnt[len - 1];
8          else if (i == cur) ans += f2(num, offset / 10, len - 1);
9          else break;
10     }
11     return ans;
12 }
```

题目3：统计整数数目

题面

给定两个数字字符串 num1、num2，以及 min_sum、max_sum。若整数 x 满足 $\text{num1} \leq x \leq \text{num2}$ 且 $\text{min_sum} \leq \text{digit_sum}(x) \leq \text{max_sum}$ ，称 x 为好整数。返回好整数个数，答案对 $1e9+7$ 取模。

- 测试链接：<https://leetcode.cn/problems/count-of-integers/>

核心思路

大策略： `count(num1, num2) = f(0→num2) - f(0→num1) + (check(num1) ? 1 : 0)`

因为 num1 是字符串，做减一操作较麻烦，所以用"不包含 num1"的方式算，最后单独检查 num1 自身。

递归 `f(i, sum, free)`:

- i: 当前处理到第几位（从高位 0 开始）
- sum: 之前决定的数字累加和
- free: 是否已确定比 num 小

剪枝:

- `sum > max_sum` → 返回 0
- `sum + (len-i)*9 < min_sum` → 即使后面全选 9 也不够，返回 0

转移:

- `free=0` (受限制) : pick 从 0 到 cur-1 全变成 free=1, pick=cur 保持 free=0
- `free=1` (自由) : pick 从 0 到 9 全部 free=1

坑点

- 前导零无所谓（不影响 digit sum），所以不需要像题目4那样区分首位
- 必须挂缓存才能过（len 可达 23，sum 可达 400）
- 每一步取模，`(ans - f() + MOD) % MOD` 防负数
- dp 每次调用前 memset 重置（因为 len/max_sum 可能不同）

代码

```
1 int f(int i, int sum, int free) {
2     if (sum > maxSum) return 0;
3     if (sum + (len - i) * 9 < minSum) return 0;
4     if (i == len) return 1;
5     if (dp[i][sum][free] != -1) return dp[i][sum][free];
6     int cur = num[i] - '0';
7     int ans = 0;
8     if (free == 0) {
9         for (int pick = 0; pick < cur; pick++)
10             ans = (ans + f(i + 1, sum + pick, 1)) % MOD;
11         ans = (ans + f(i + 1, sum + cur, 0)) % MOD;
12     } else {
13         for (int pick = 0; pick <= 9; pick++)
14             ans = (ans + f(i + 1, sum + pick, 1)) % MOD;
15     }
16     return dp[i][sum][free] = ans;
17 }
```

题目4：完全没有重复的数字个数

题面

给定正整数 n ，返回 $[1, n]$ 范围内每一位都互不相同的正整数个数。

- 测试链接：<https://leetcode.cn/problems/count-special-integers/>

核心思路

分两块计算：

第一块：位数少于 len 的所有情况（排列组合）

- 1 位：9 (1~9)
- 2 位： 9×9
- 3 位： $9 \times 9 \times 8$
- ...累加到 len-1 位

第二块：位数 = len 且 $\leq n$ 的情况

- 预处理 `cnt[i]`：已确定前缀不为空，剩 i 位待选时的排列数

- 例 len=4: cnt[3]=9×8×7, cnt[2]=8×7, cnt[1]=7, cnt[0]=1
- 首位 < n 首位: $(\text{first}-1) \times \text{cnt}[\text{len}-1]$
- 首位 = n 首位: 递归 f(len-1, offset/10, status)
- 递归 f(len, offset, status):** status 用位图标记 0~9 哪些数字已使用。
- 遍历 0 到 cur-1, 若未用过 → $\text{ans} += \text{cnt}[\text{len}-1]$ (直接结算)
- 若 cur 未用过 → 递归 f(len-1, offset/10, status | (1<<cur))
- 若 cur 用过 → 自身非法, 只算比它小的

坑点

- 递归前提是"已确定前缀不为空", 这样后续位可以选 0
- $n \leq 10^9$ (最多 9 位), 不挂缓存也能过
- 挂缓存则状态数 = $\text{len} \times 2^{10} \approx 9 \times 1024$, 每个状态枚举 10 → $O(9 \times 1024 \times 10)$
- 注意 cnt 表的计算依赖 len (不同 len 的 cnt 不同)

代码

```

1  int f(int num, int len, int offset, int status) {
2      if (len == 0) return 1; // num 自己
3      int ans = 0;
4      int first = (num / offset) % 10;
5      for (int cur = 0; cur < first; cur++) {
6          if ((status & (1 << cur)) == 0)
7              ans += cnt[len - 1];
8      }
9      if ((status & (1 << first)) == 0)
10         ans += f(num, len - 1, offset / 10, status | (1 << first));
11     return ans;
12 }

```

题目5: 至少有1位重复的数字个数

题面

给定正整数 n, 返回 [1, n] 范围内至少有 1 位重复数字的正整数个数。

- 测试链接: <https://leetcode.cn/problems/numbers-with-repeated-digits/>

核心思路

直接用题目4的结果: $\text{numDupDigitsAtMostN}(n) = n - \text{countSpecialNumbers}(n)$

"每位都不同"的补集就是"至少有一位重复"。

坑点

- 直接复用题目4的 countSpecialNumbers 即可
- 不需要额外递归

代码

```
1 int numDupDigitsAtMostN(int n) {  
2     return n - countSpecialNumbers(n);  
3 }
```

数位DP通用模板

```
1 递归 f(i, 状态...):  
2     1. base case: i == len → 返回 1 (前缀本身是有效答案)  
3     2. 剪枝条件 (如累加和超限)  
4     3. 如果已缓存 → 返回缓存  
5     4. cur = num[i] - '0'  
6     5. 分支:  
7         - free=0: pick 0..cur-1 → free=1; pick=cur → free=0  
8         - free=1: pick 0..9 → 全 free=1  
9     6. 存入缓存, 返回结果
```

关键点:

- 大多数数位 DP 位数少 (≤ 10)，裸递归能过
- 位数多时挂缓存即可 (状态数 = 位数 $\times$ 少量标志位)
- offset 只是辅助提取数字，非关键参数
- 灵活使用预处理表 (排列组合) 加速结算